

Rilevazione dell'opinione delle/degli studentesse/studenti

- Importantissimo ottenere una compilazione attenta e consapevole da parte degli studenti e delle studentesse
 - Feedback per migliorare il corso nelle prossime edizioni
 - Segnalazione di eventuali problemi, per facilitarne l'identificazione e la soluzione
- Superati I 2/3 dell'insegnamento
- Tramite app (smartphone o tablet) oppure online (esse3 – questionnaire)
- 10 minuti (incluso download app)
- Obbligatoria prima del primo appello d'esame
 - Tutte le domande sono obbligatorie
 - prevista opzione di risposta “Non Rispondo”

UML 2.0: COMPONENT, PACKAGE AND DEPLOYMENT DIAGRAM

Ingegneria del Software a.a. 2025-26

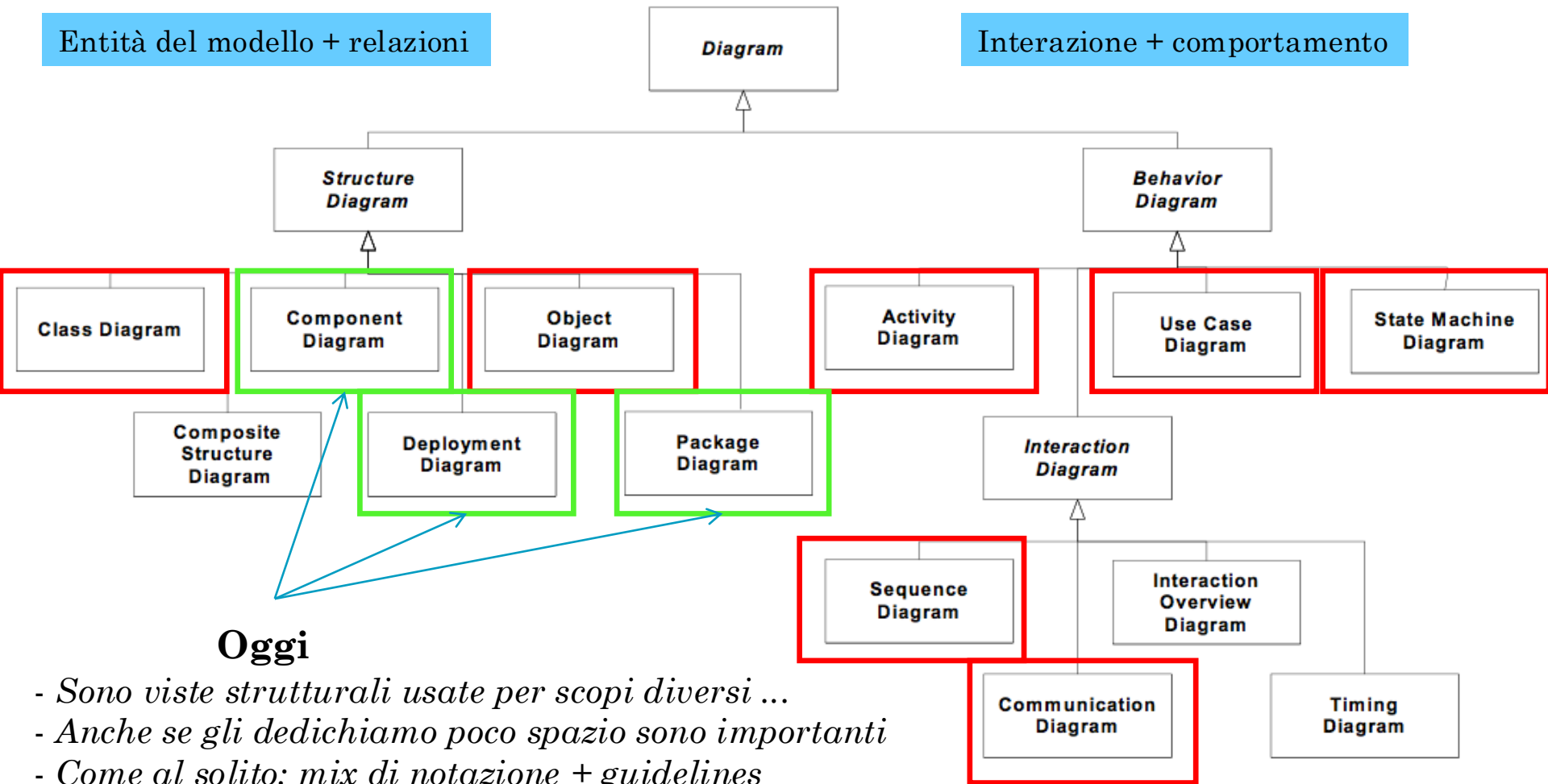
ORIENTIAMOCI ...

Modellazione statica

Entità del modello + relazioni

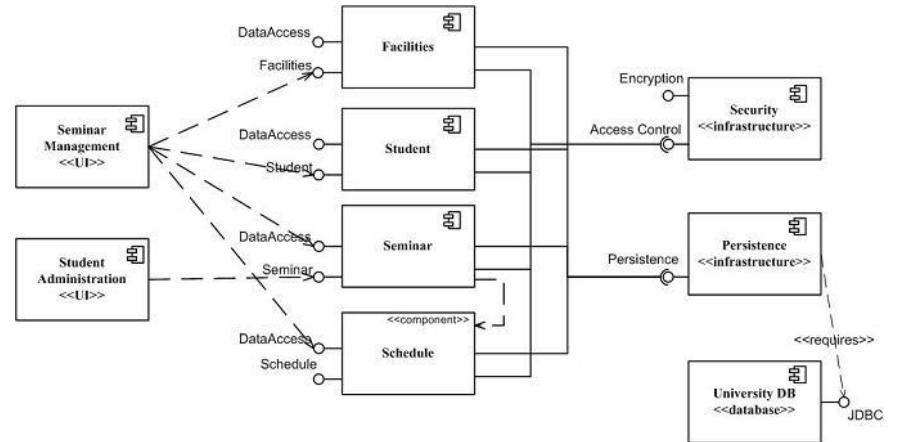
Modellazione dinamica

Interazione + comportamento



Oggi

- Sono viste strutturali usate per scopi diversi ...
- Anche se gli dedichiamo poco spazio sono importanti
- Come al solito: mix di notazione + guidelines



COMPONENT DIAGRAM

CONCETTO DI COMPONENTE IN UML

- Il termine componente assume un significato (leggermente) diverso a seconda della piattaforma, ambiente, sistema che consideriamo
 - .Net, Java world, ...
- In UML un componente è un concetto molto generale:
“Modular unit with well-defined interfaces that is replaceable within its environment”
- Una “**scatola nera**” il cui comportamento esterno è completamente definito dalle sue interfacce
 - Sono **rimpiazzabili** e **componibili**!



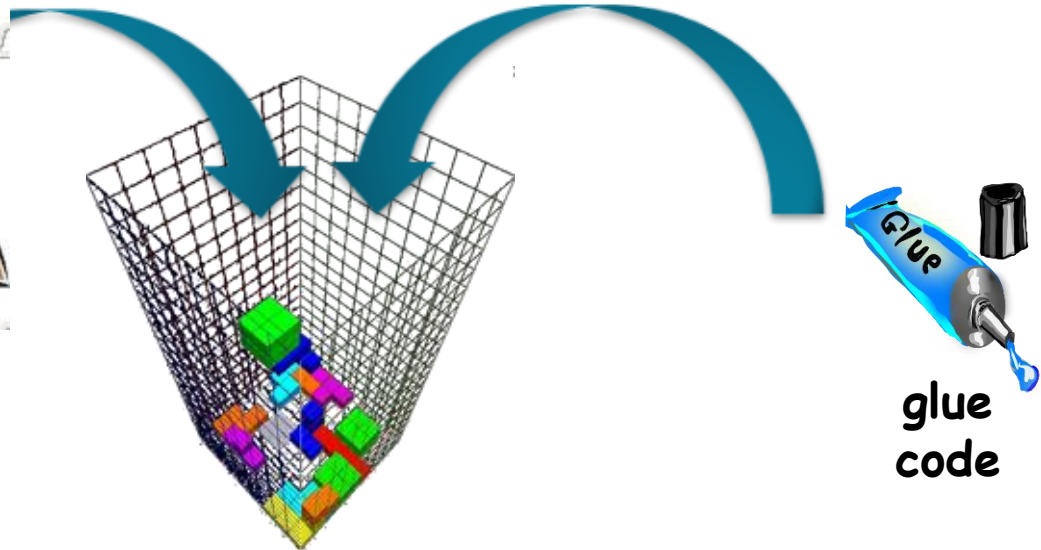
**Components =
LEGO bricks**

SVILUPPO BASATO SUI COMPONENTI

component marketplace



components

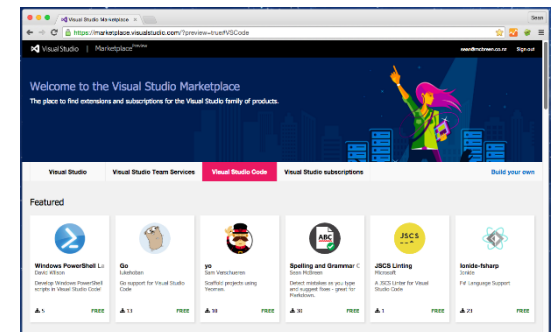


glue
code

composition
framework
("plumbing"
or
infrastructure)



working
systems

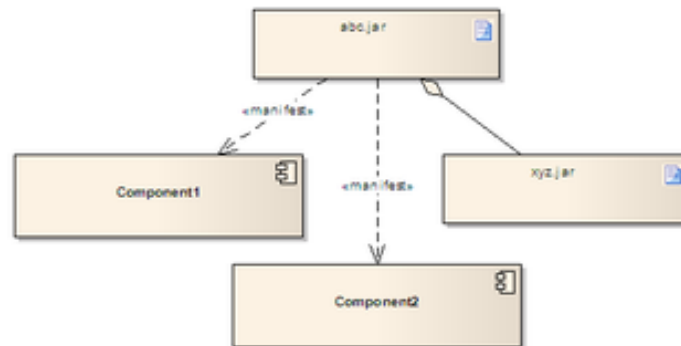


COMPONENTI E ARTEFATTI

- I componenti sono **entità logiche** che sono realizzate da **artefatti** (che sono invece entità fisiche)

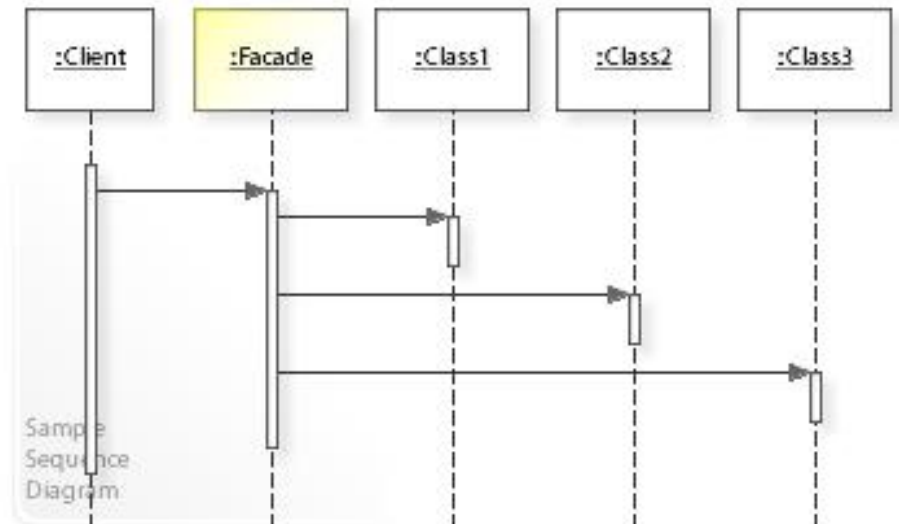
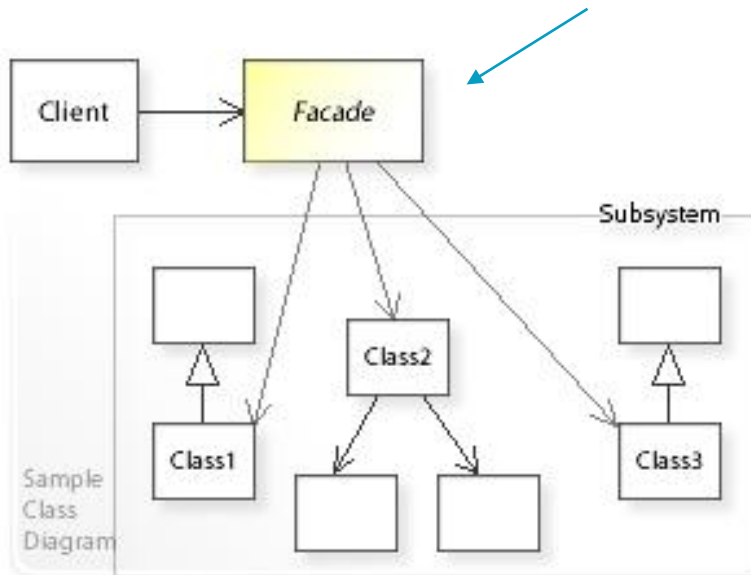
Es.

- **Sottosistema** realizzato tramite un insieme di classi (file JAR)
- **Componente Webratio** realizzato tramite un file JAR
- **Servizio Web REST** realizzato tramite Jersey / JAX-RS
- **Microservice** realizzato con Spring Boot
- **EmailVerify for .NET** contenuto in un .msi package



SOTTOSISTEMA

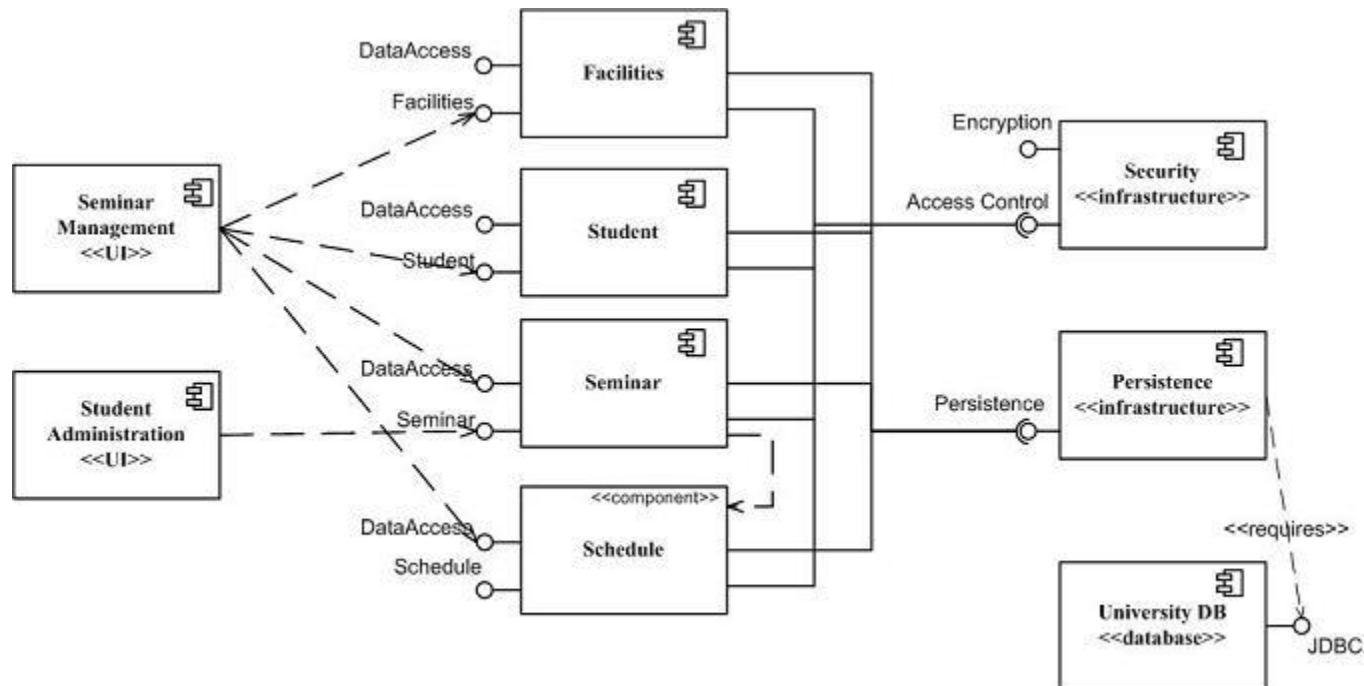
Espone l'interfaccia



Sottoparte di un sistema che può essere eseguita anche a se stante

DIAGRAMMA DELLE COMPONENTI UML

- Mostrano le componenti che costituiscono il sistema e le dipendenze tra di esse
- Sono utili per **rappresentare l'architettura software (High level design)** di un sistema



COMPONENTI VS. CLASSI

- Perché si usano le componenti per descrivere l'architettura di un sistema SW?
- Le classi sono componenti di “**grana troppo fine**” per dare una buona panoramica del sistema

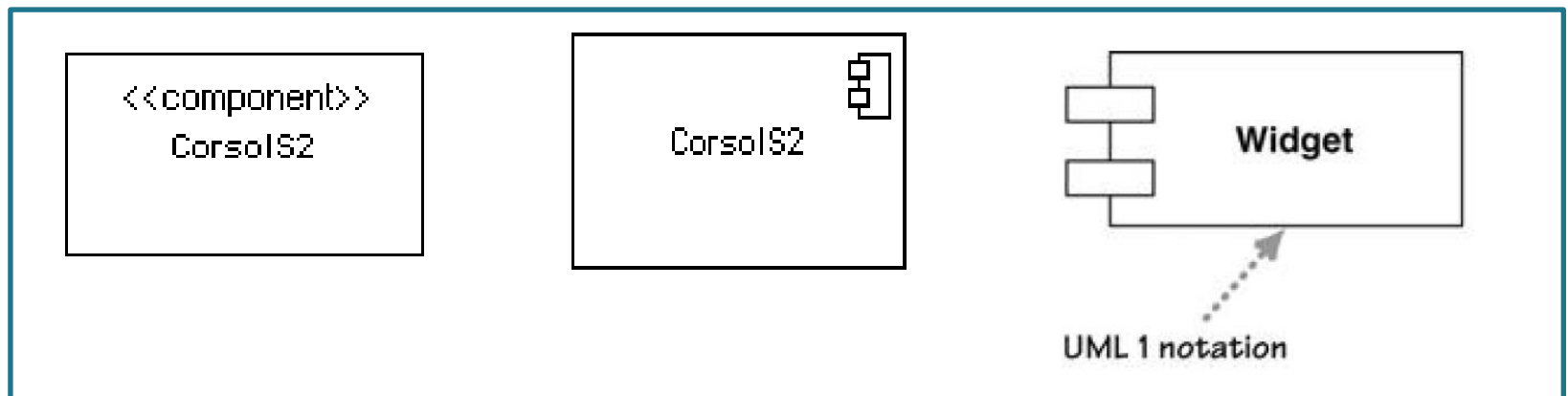
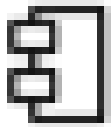
Guardando gli alberi si finisce di perdere di vista il bosco!!!



NOTAZIONE (NON PROLISSA)

- Un componente è mostrato come un rettangolo con:

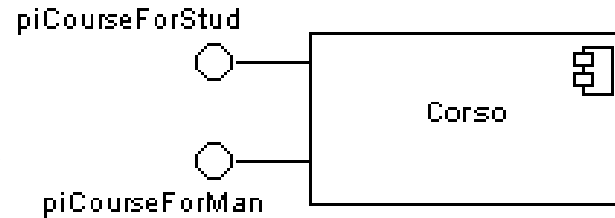
- Parola chiave <<component>> oppure Icona
- Nome



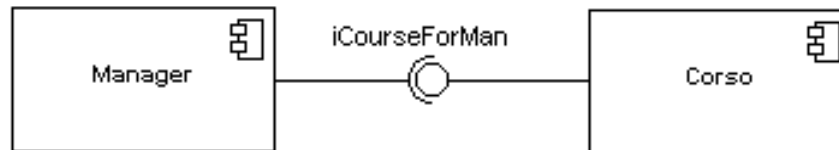
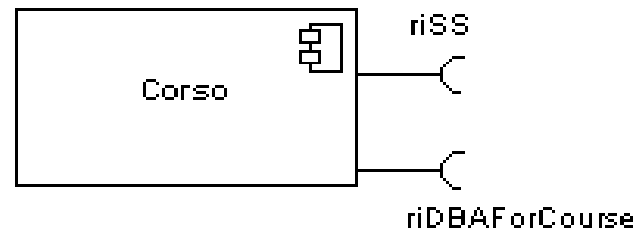
- Esistono parole chiave più specifiche come:
 - <<Web service>>, <<Microservice>>, <<Subsystem>>, ...
 - In più se ne possono sempre aggiungere

INTERFACCE

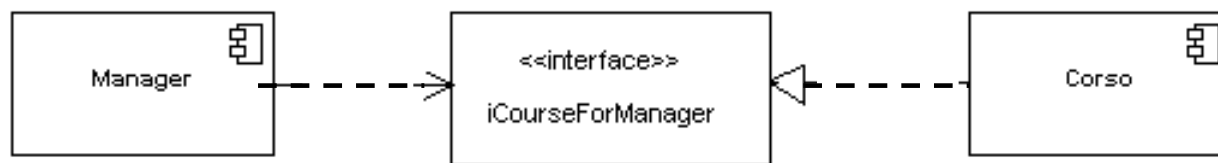
- Fornite (**lollipop**)



- Richieste (**socket**)



L'interfaccia iCourseForMan **assembla** le due componenti

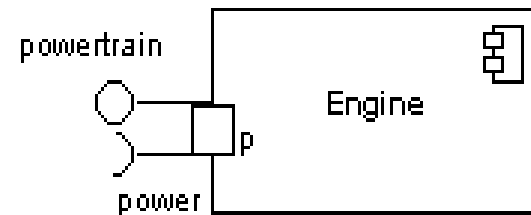
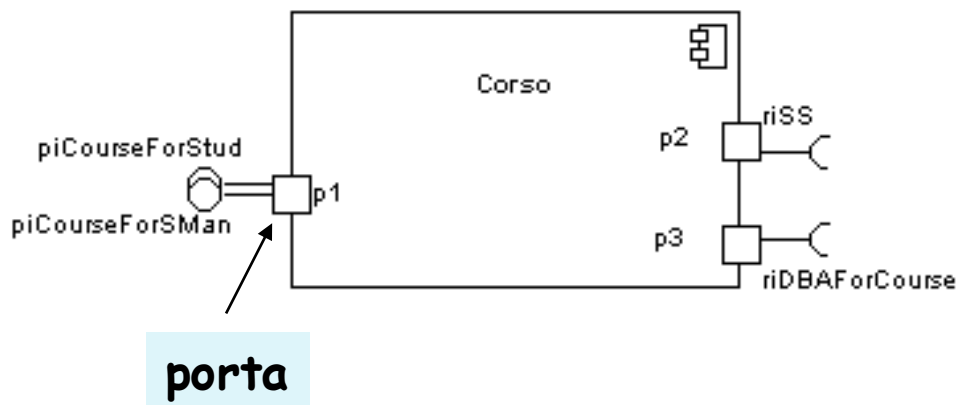


richiede/usa

fornisce/implementa

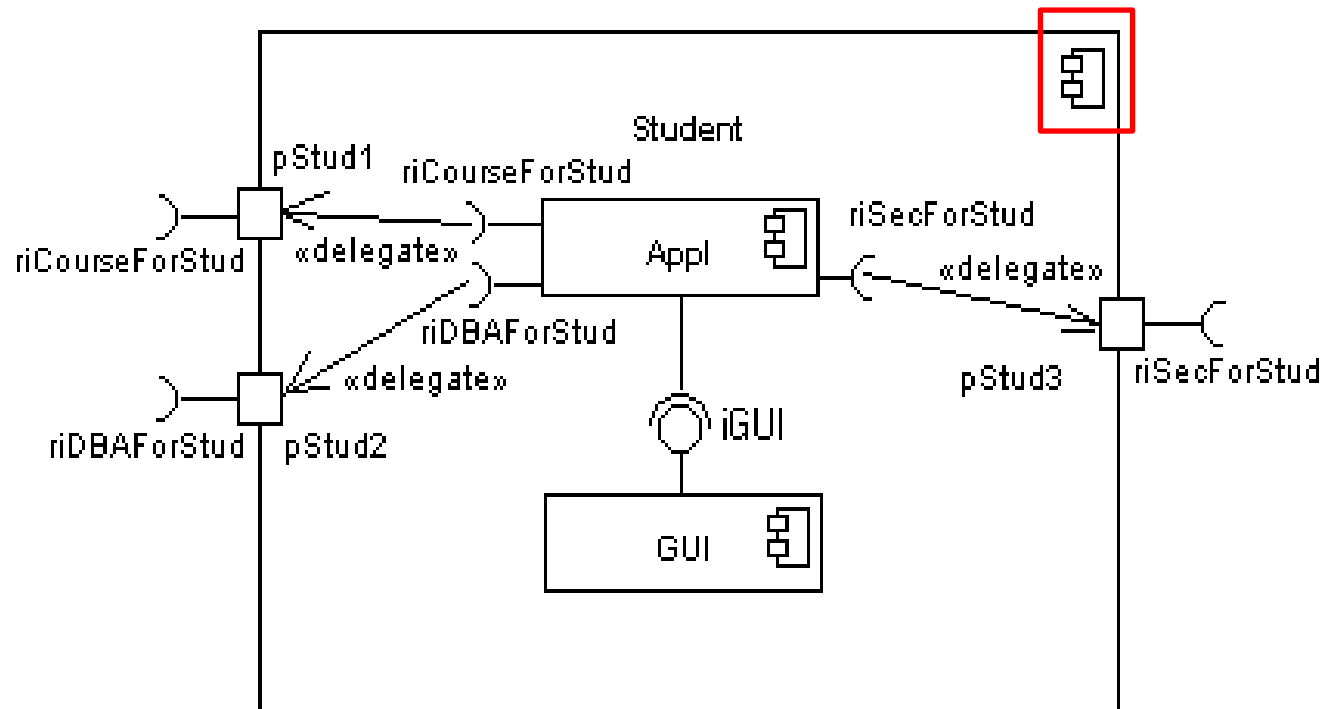
PORTE

- Una porta raggruppa un insieme “**semanticamente coeso di interfacce**”
- E’ un punto specifico di interazione tra componente-esterno e componente-interno
- Possono avere un nome (es. p1, p2, p3)



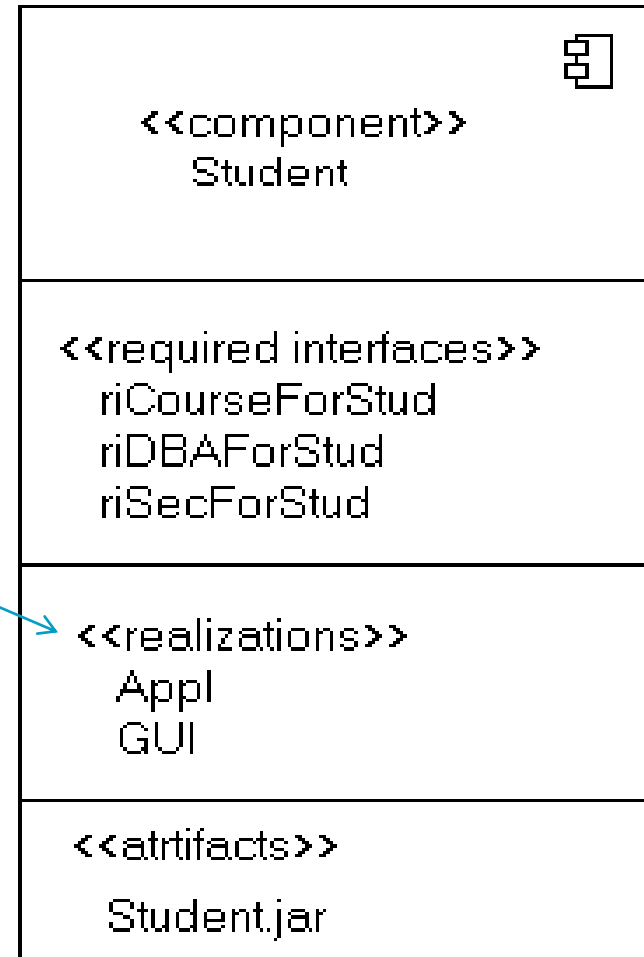
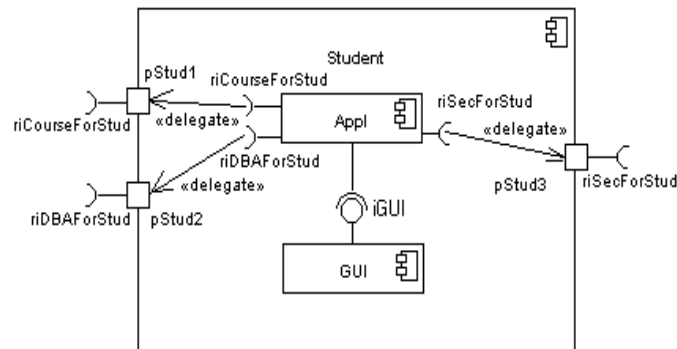
VISTA INTERNA

- Specifica la **struttura interna del componente**
 - Utile per chi deve implementarlo
- I componenti possono essere costruiti in modo ricorsivo



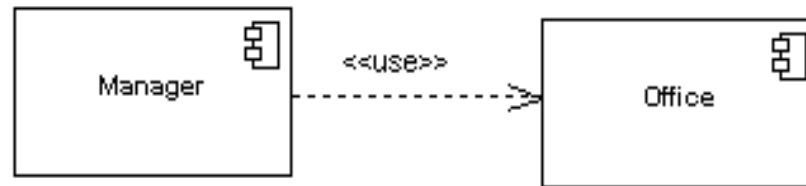
IN MODO ‘SUPER’ PROLISSO ...

- Parola chiave <<component>>
- Icona
- Nome
- Interfacce richieste/fornite
- Struttura interna
- Artefatti
 - File che realizzano il componente



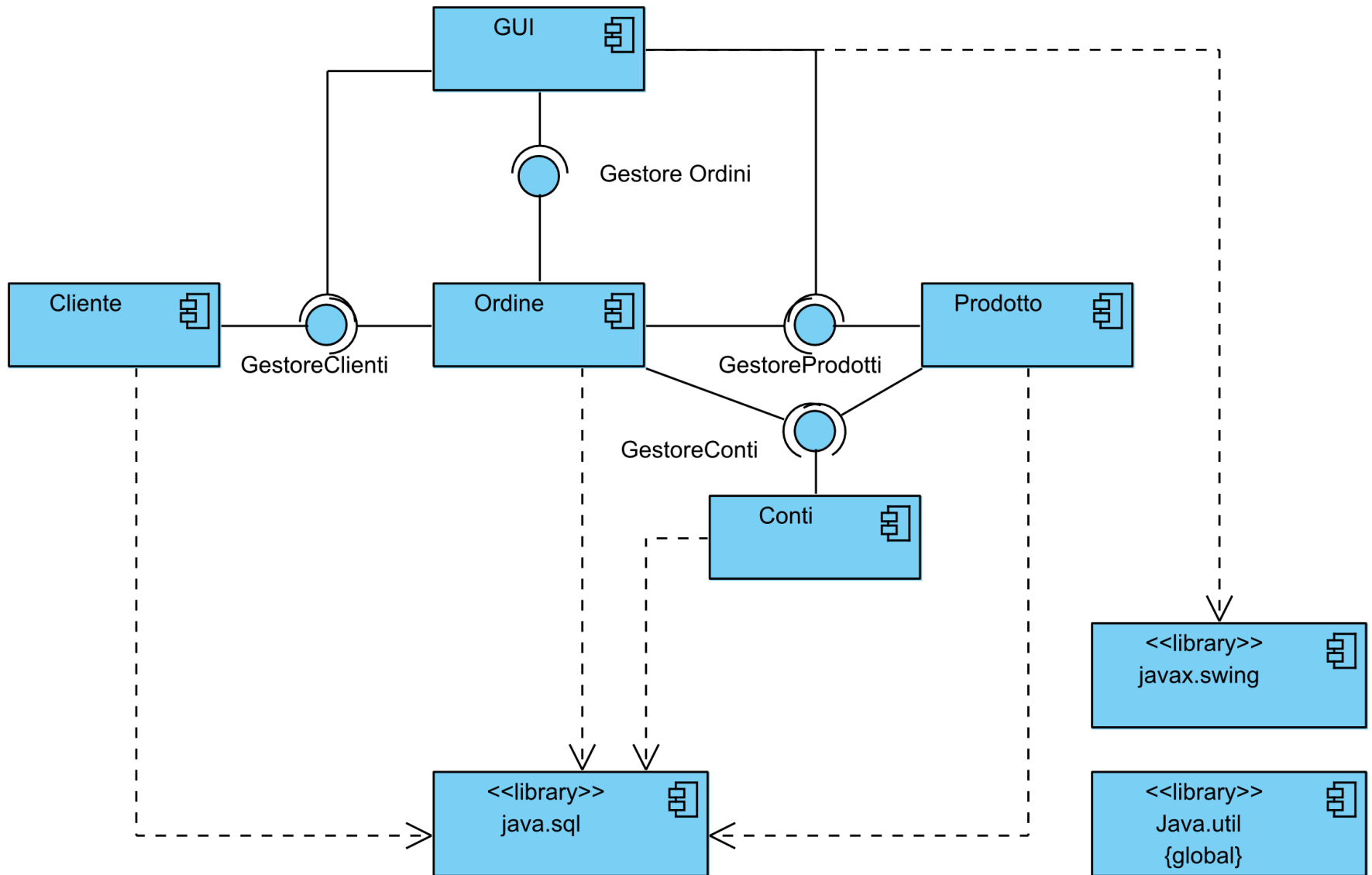
DIPENDENZA TRA COMPONENTI

- I componenti possono essere connessi con la relazione di dipendenza

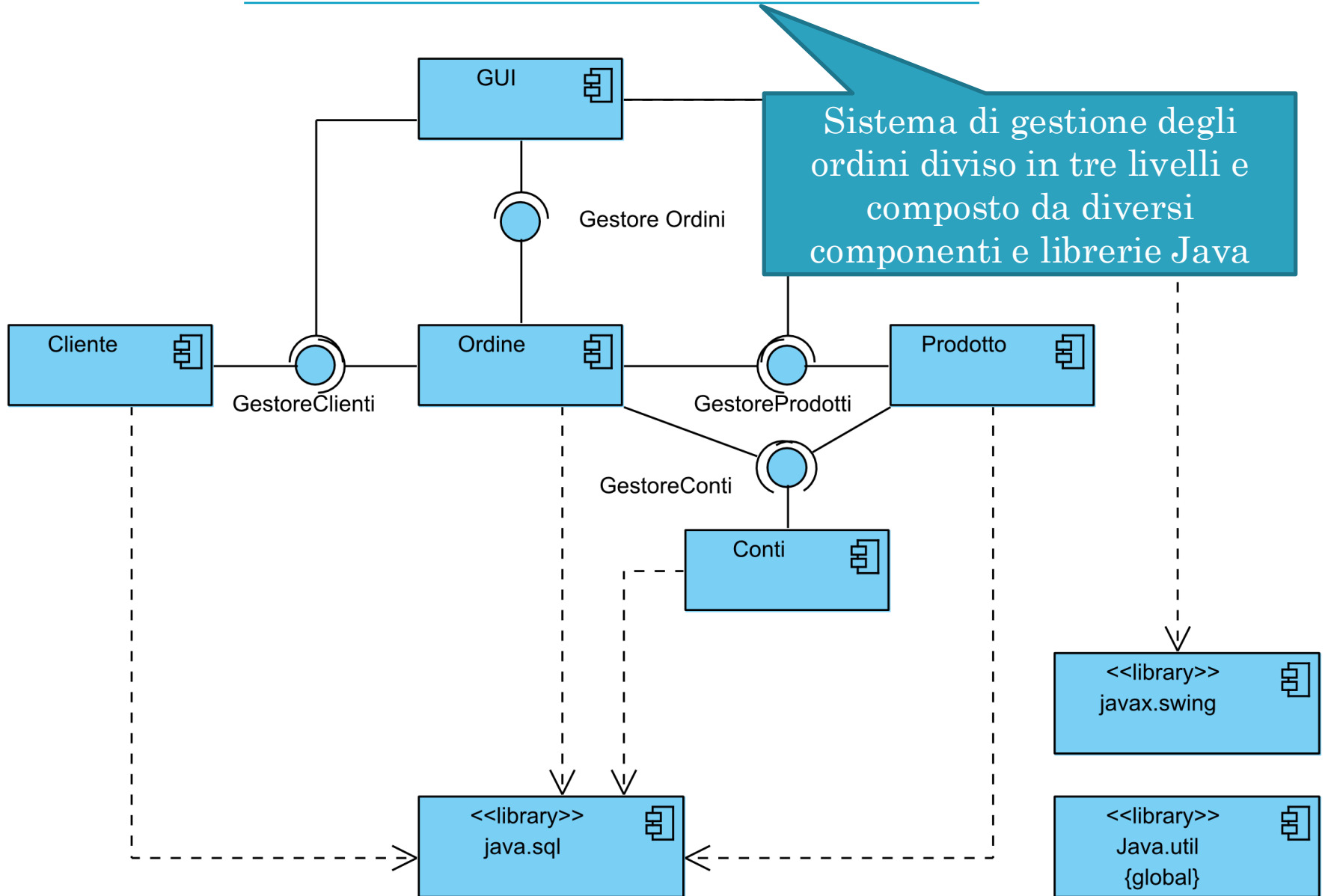


- Solito significato: il componente **Manager** ha bisogno di **Office** per poter funzionare
- Spesso usato quando sono utilizzate più interfacce ma non si vuole specificare quali sono ...
 - Altrimenti si usa **lollipop** e **socket**

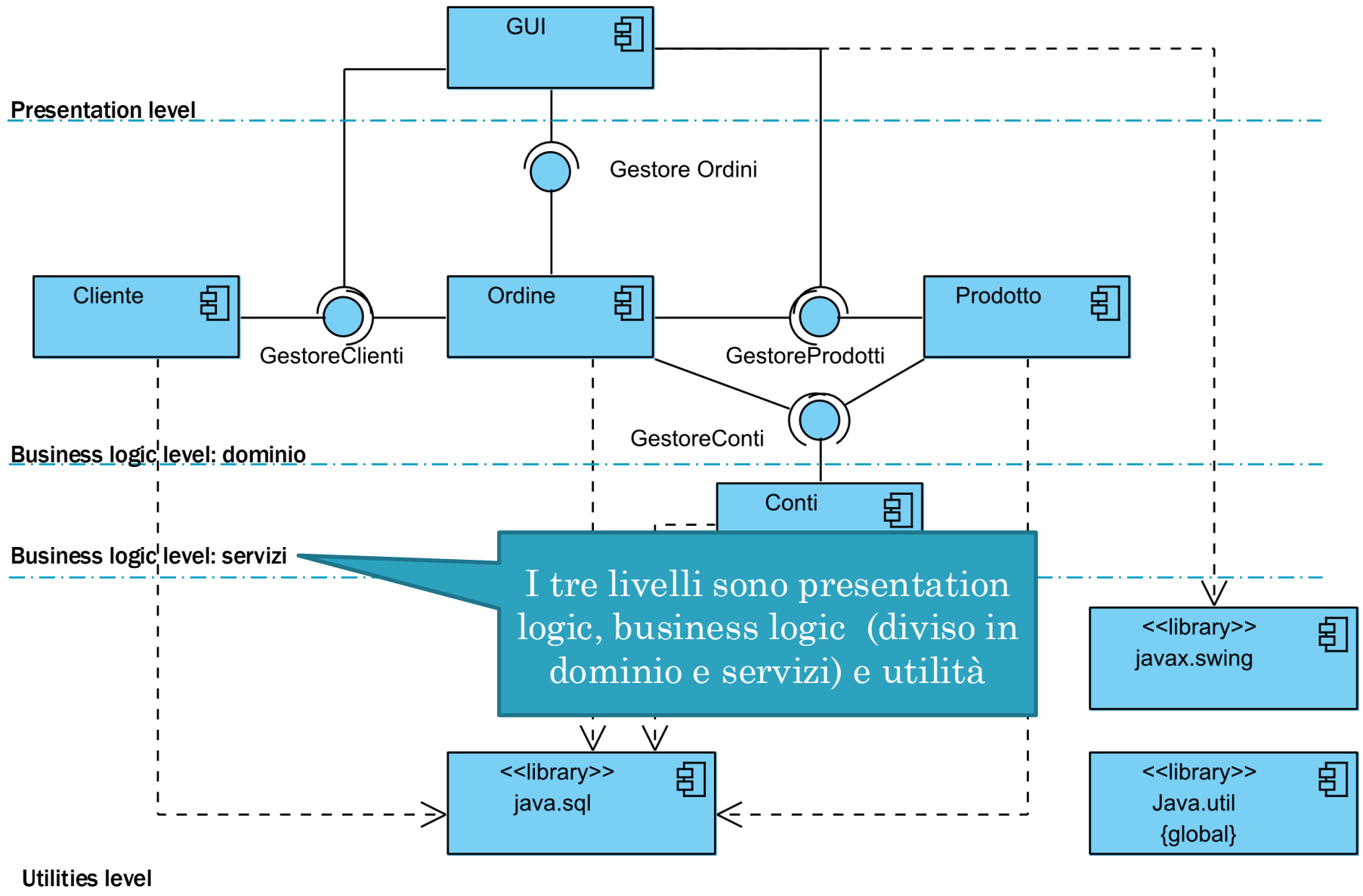
ESEMPIO: SISTEMA GESTIONE ORDINI



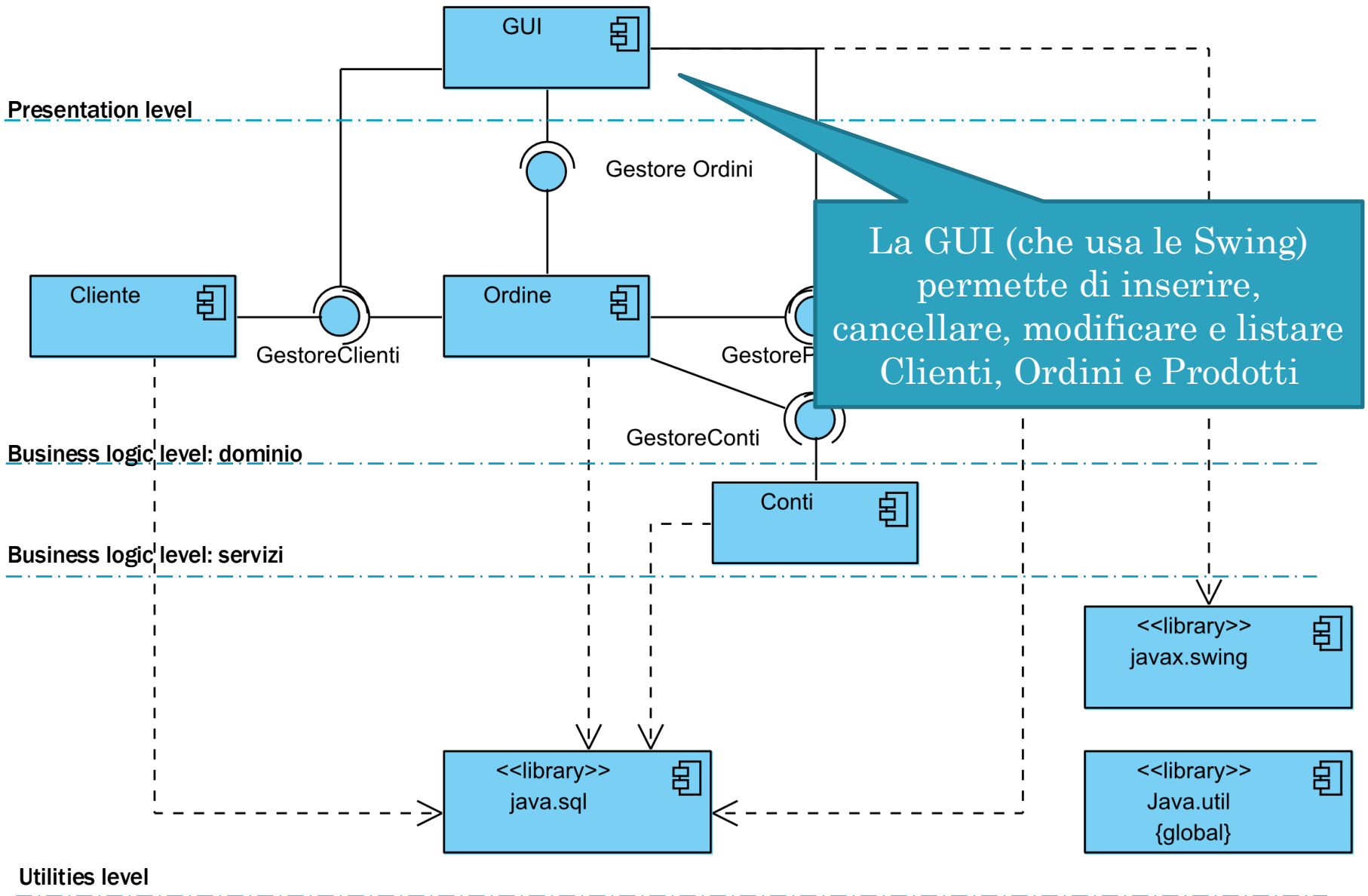
ESEMPIO: SISTEMA GESTIONE ORDINI



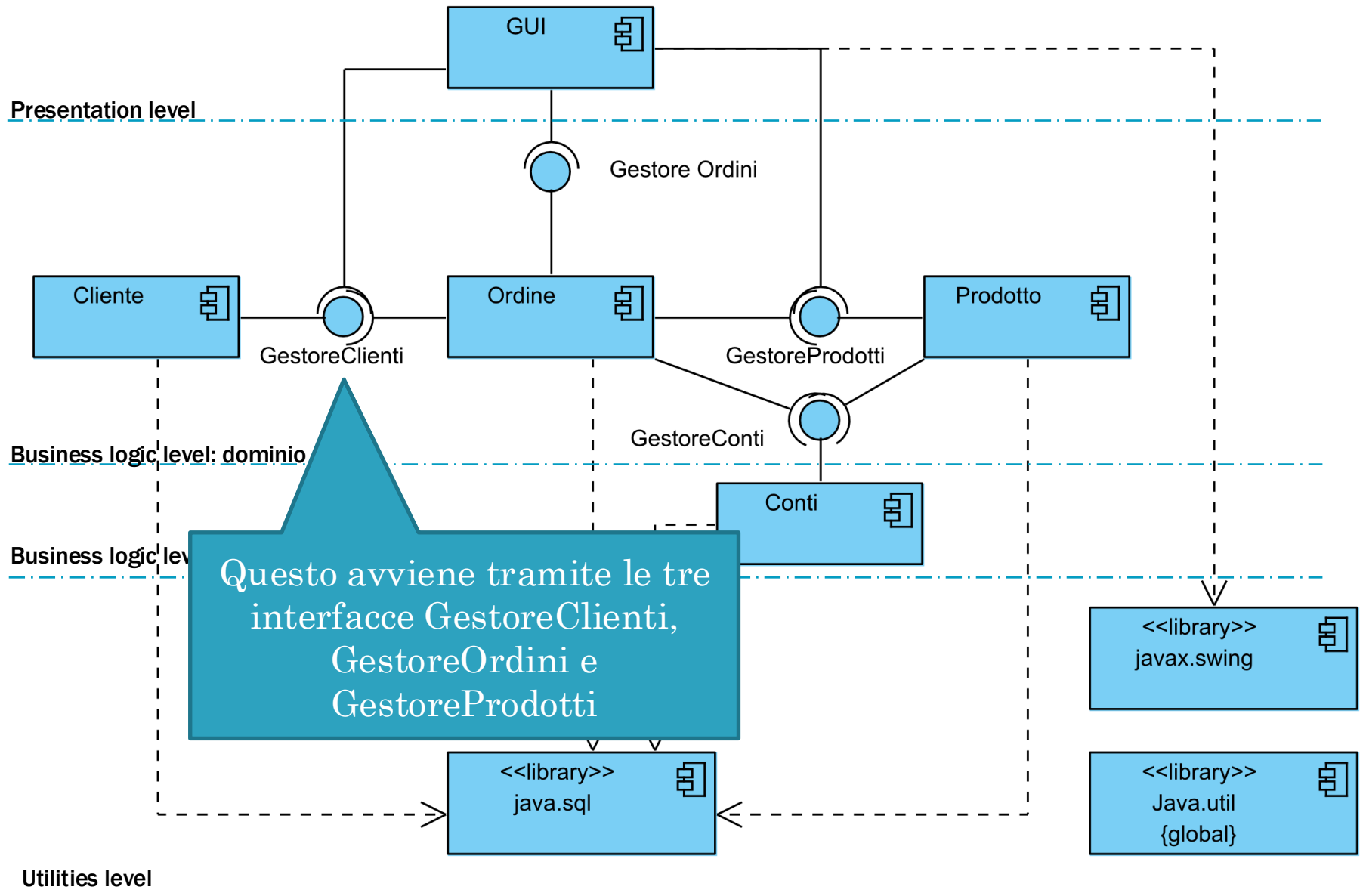
ESEMPIO: SISTEMA GESTIONE ORDINI



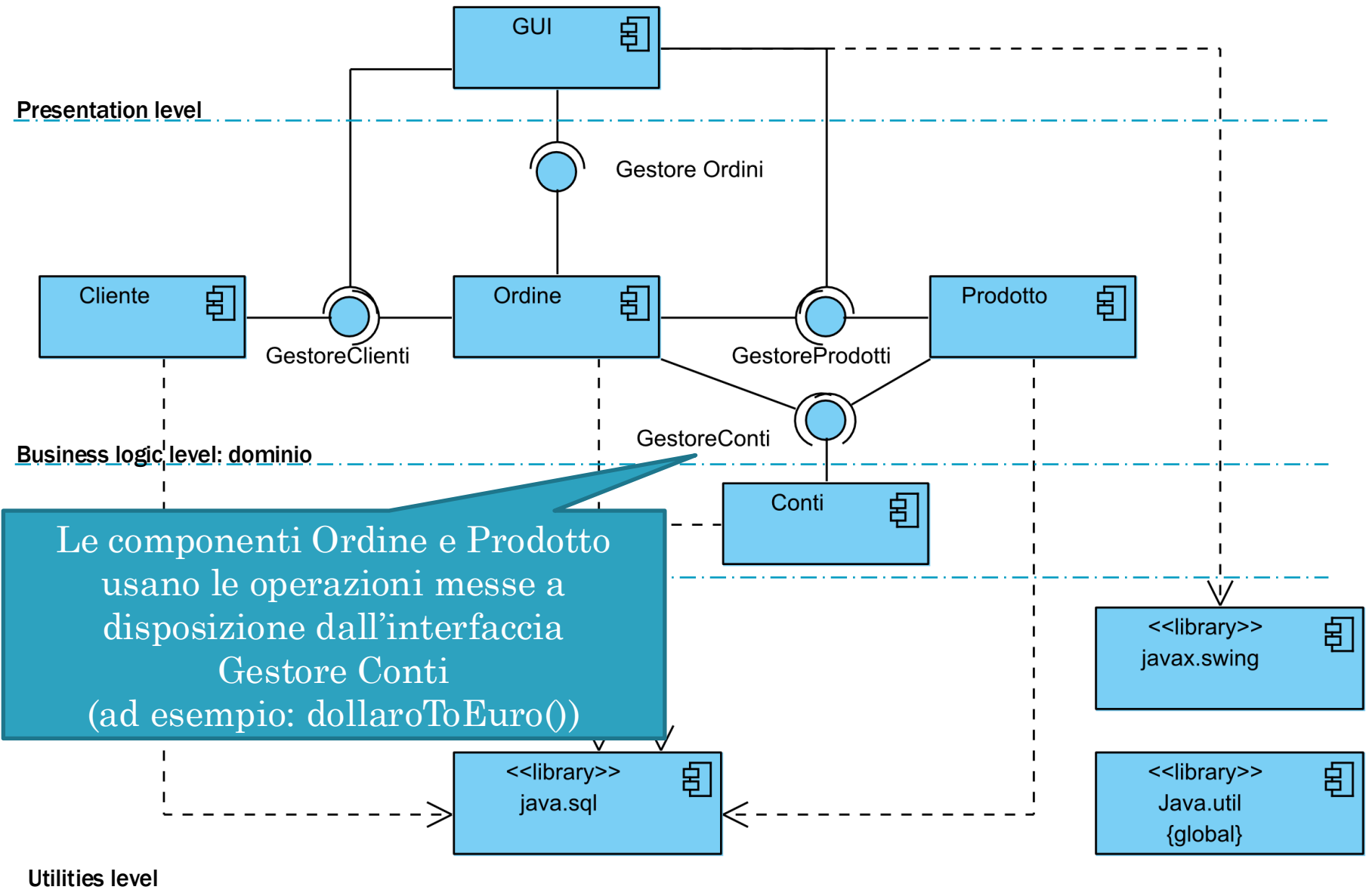
ESEMPIO: SISTEMA GESTIONE ORDINI



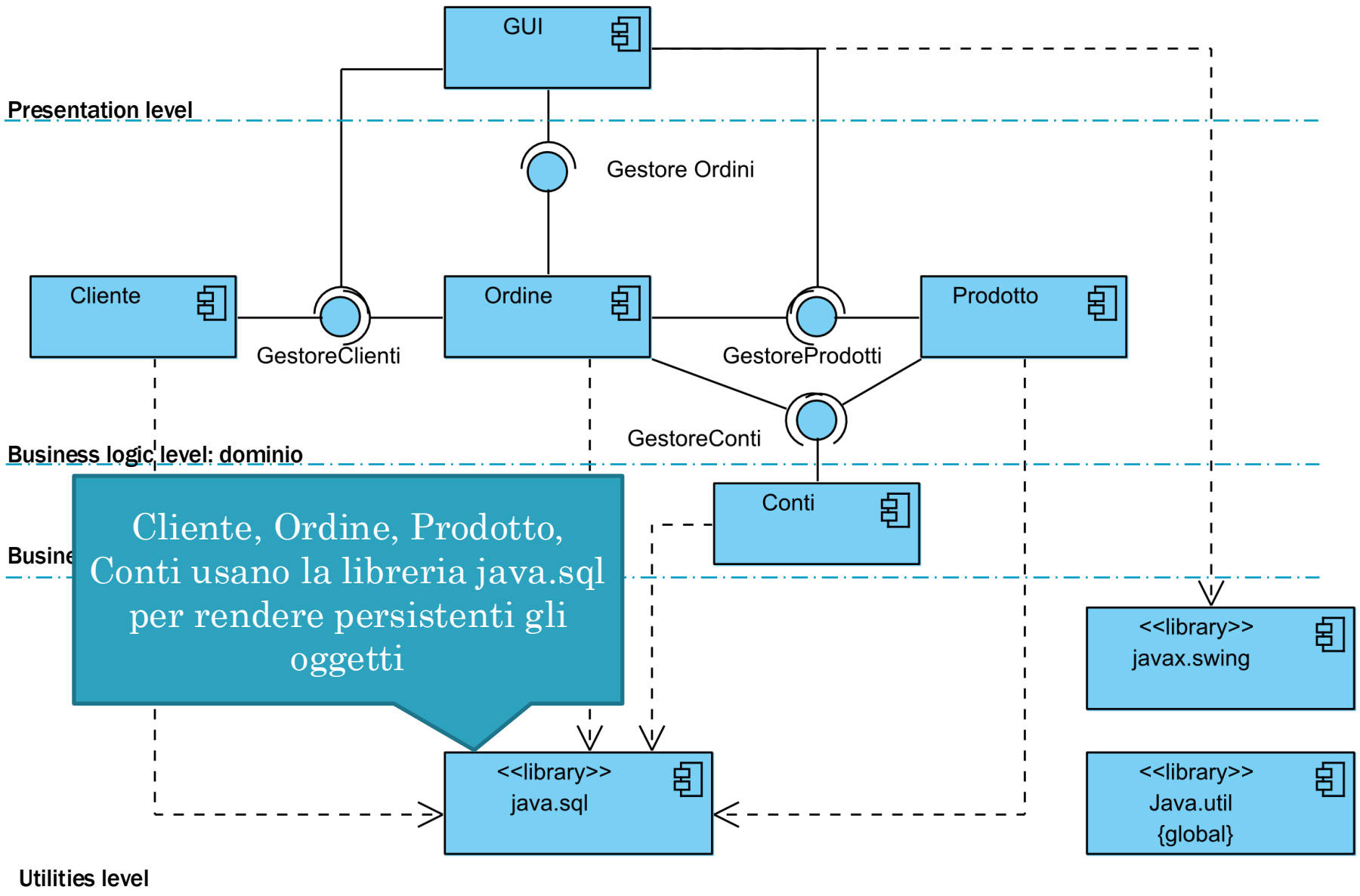
ESEMPIO: SISTEMA GESTIONE ORDINI



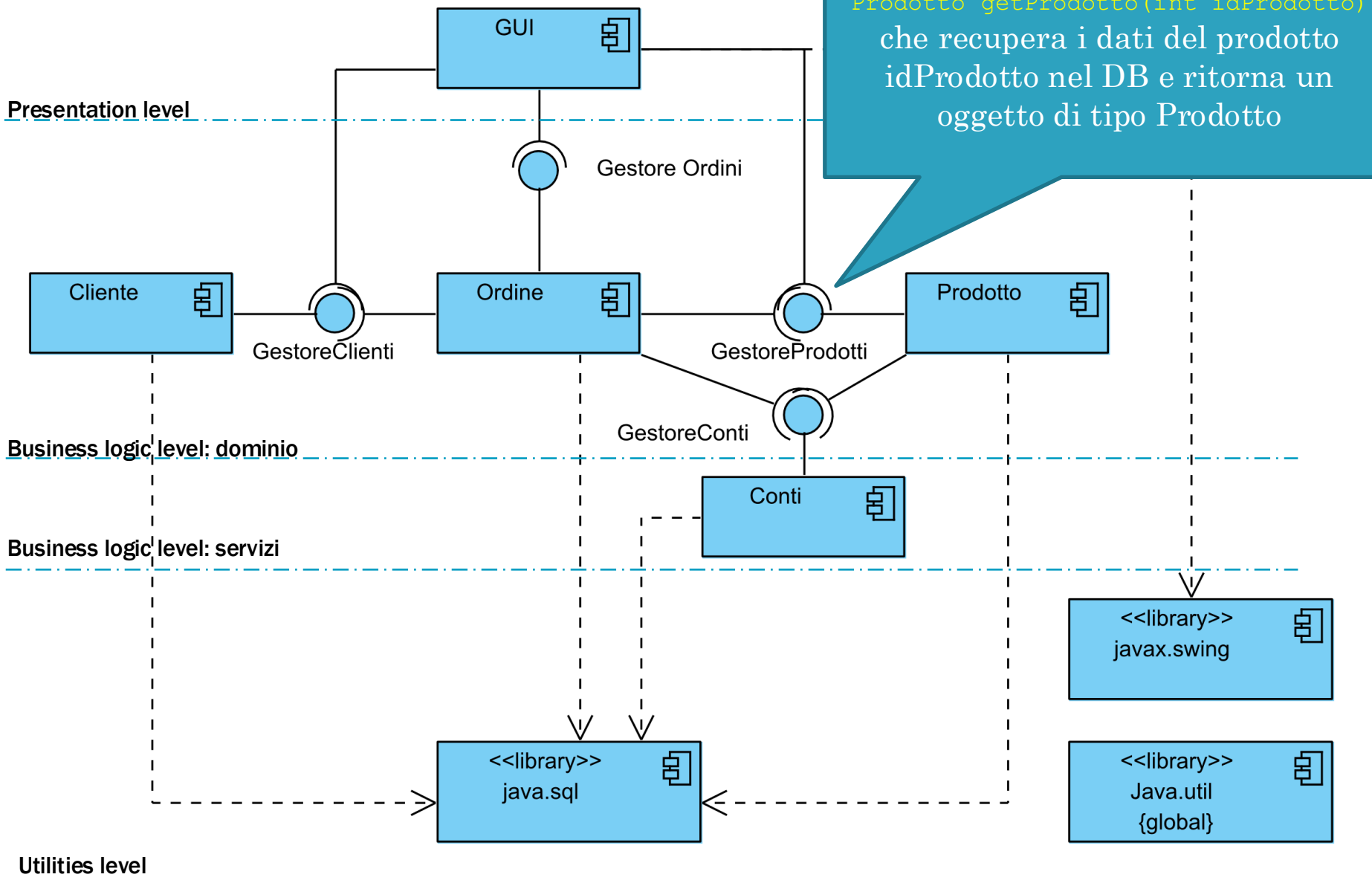
ESEMPIO: SISTEMA GESTIONE ORDINI



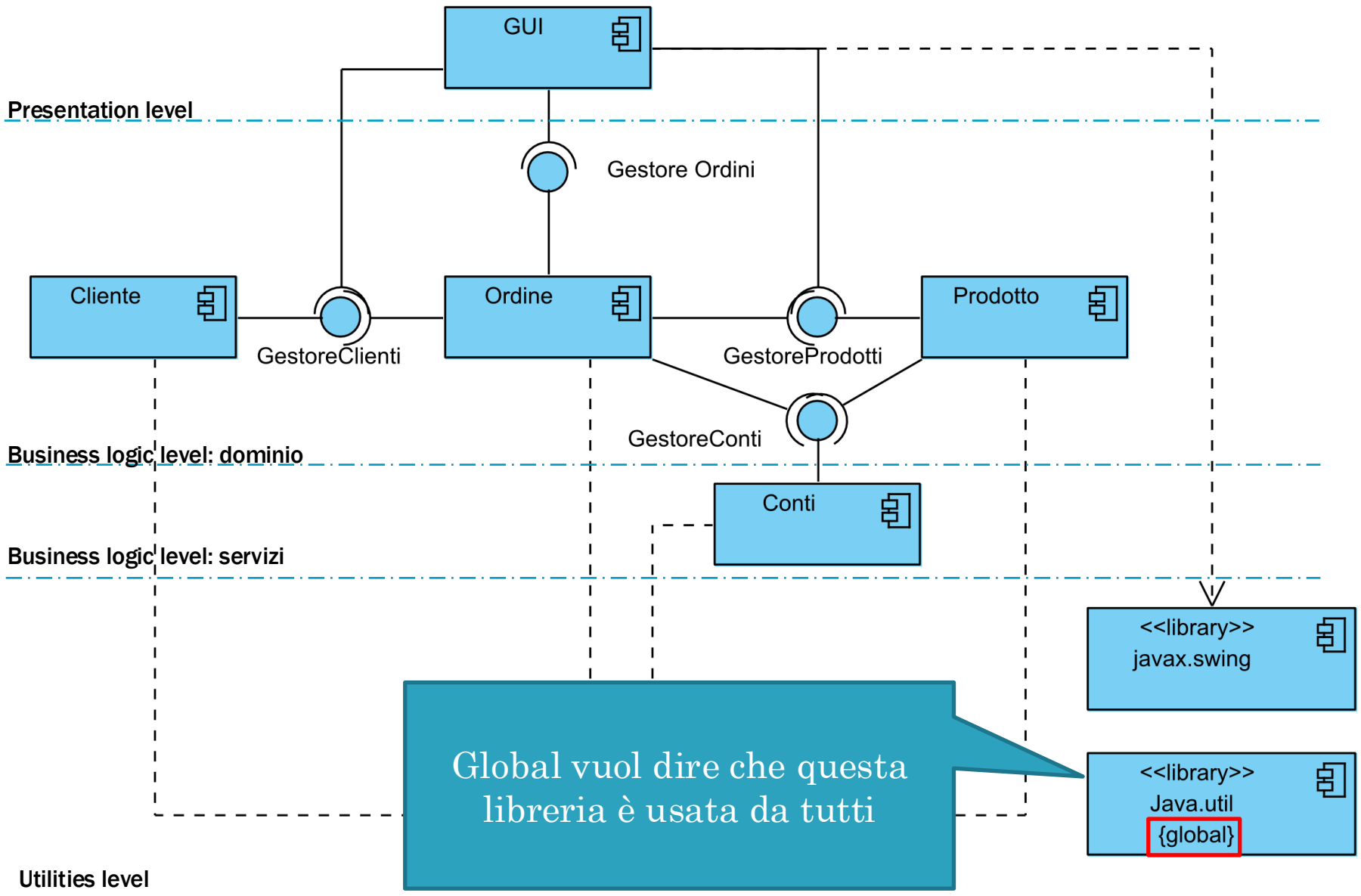
ESEMPIO: SISTEMA GESTIONE ORDINI

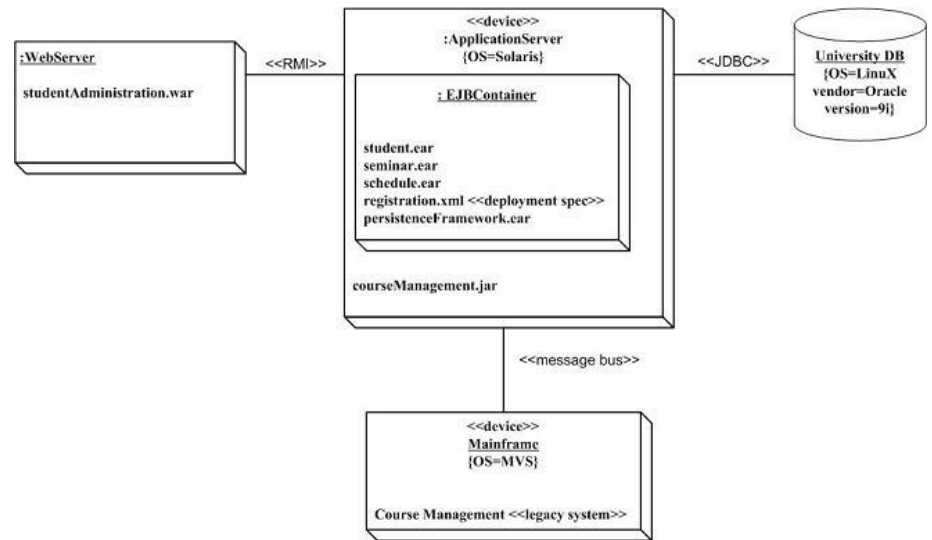


ESEMPIO: SISTEMA GESTIONE



ESEMPIO: SISTEMA GESTIONE ORDINI

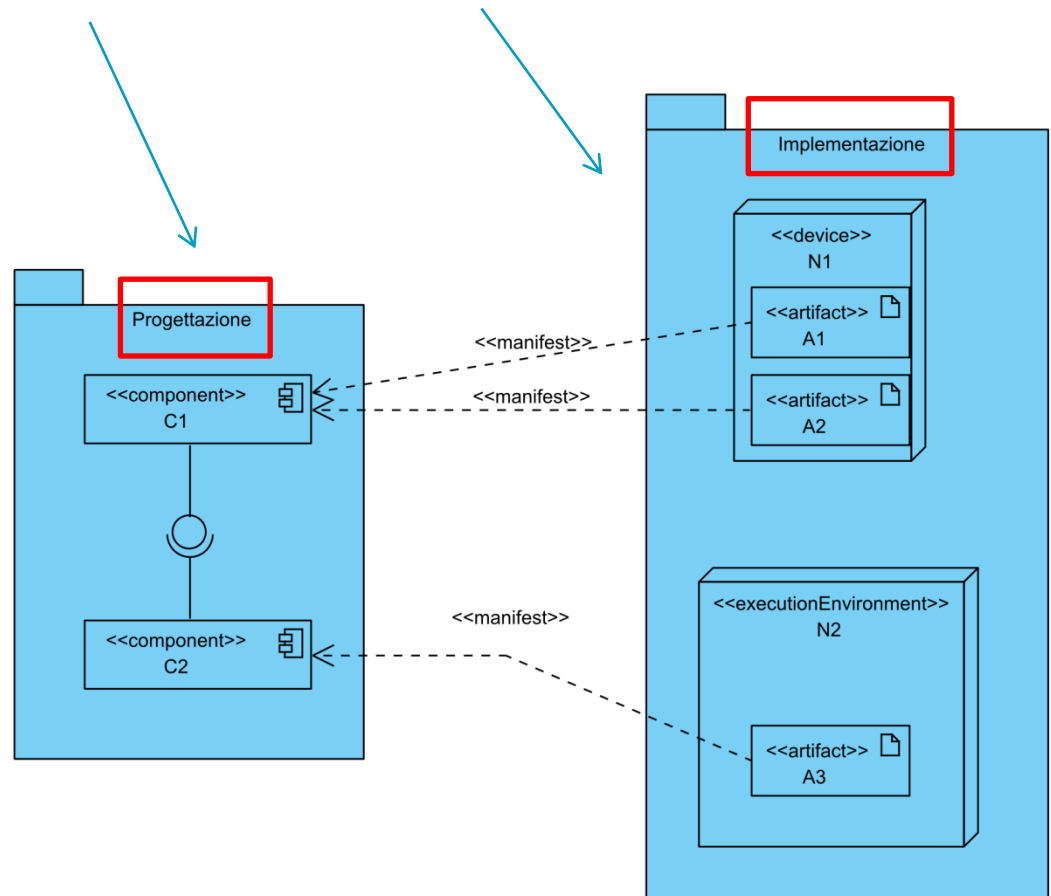




DEPLOYMENT DIAGRAM

DEPLOYMENT DIAGRAM

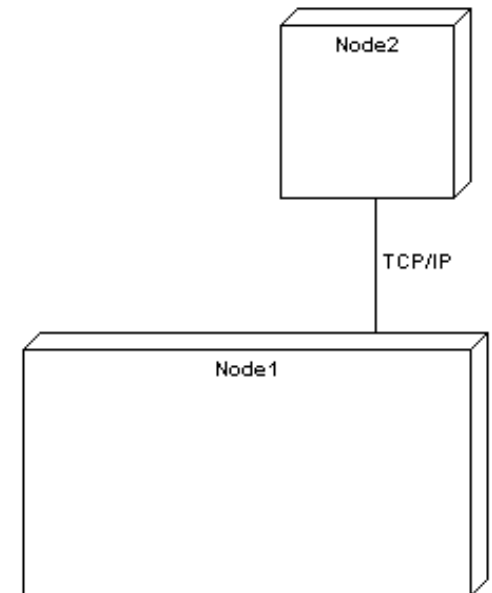
- Vista molto implementativa
- Mostra la **relazione tra hardware e software** in un sistema
- Forte link tra **component** e **deployment diagram**



Ogni nodo rappresenta "qualcosa" che può "ospitare" del software

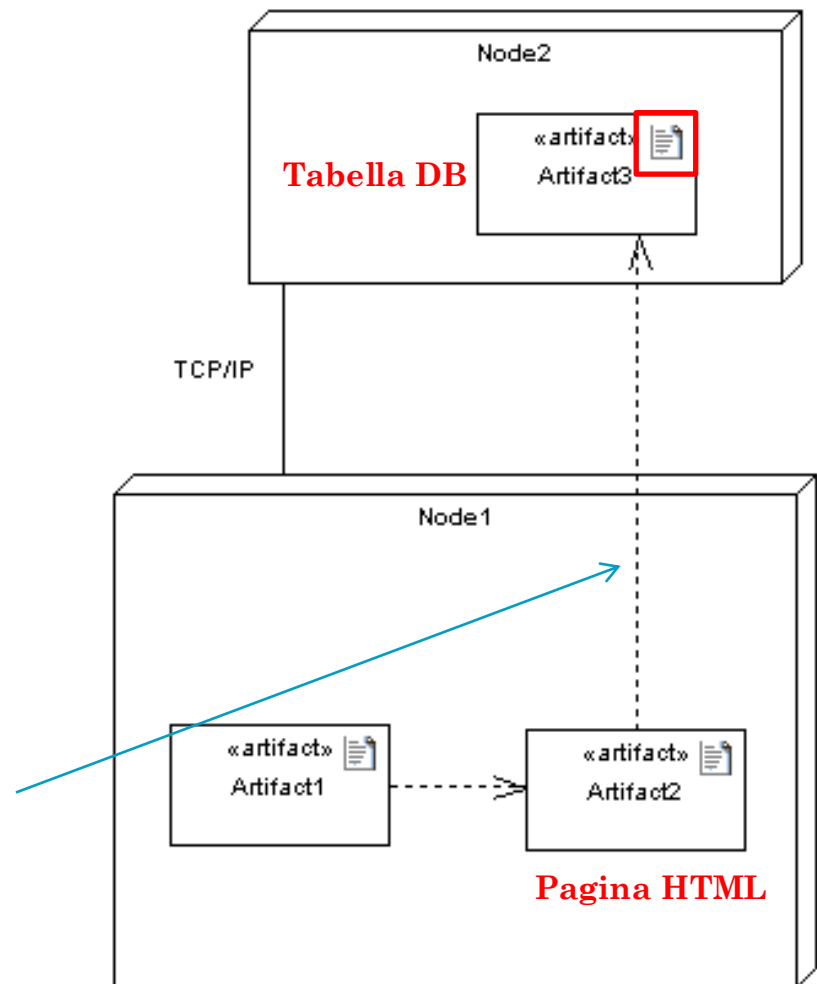
DEPLOYMENT DIAGRAM (NODI E CONNESSIONI)

- Contiene **nodi** e **connessioni**
- Un nodo rappresenta un tipo di **risorsa computazionale**
 - Una periferica fisica (**device**)
 - per esempio un PC o un server
 - Un'**ambiente software di esecuzione**
 - per esempio un browser, una macchina virtuale, o un Docker container ...
- Una connessione tra nodi rappresenta un **canale di comunicazione** attraverso cui possono passare delle informazioni
 - Usualmente si indica il tipo di connessione
 - Es. TCP/IP



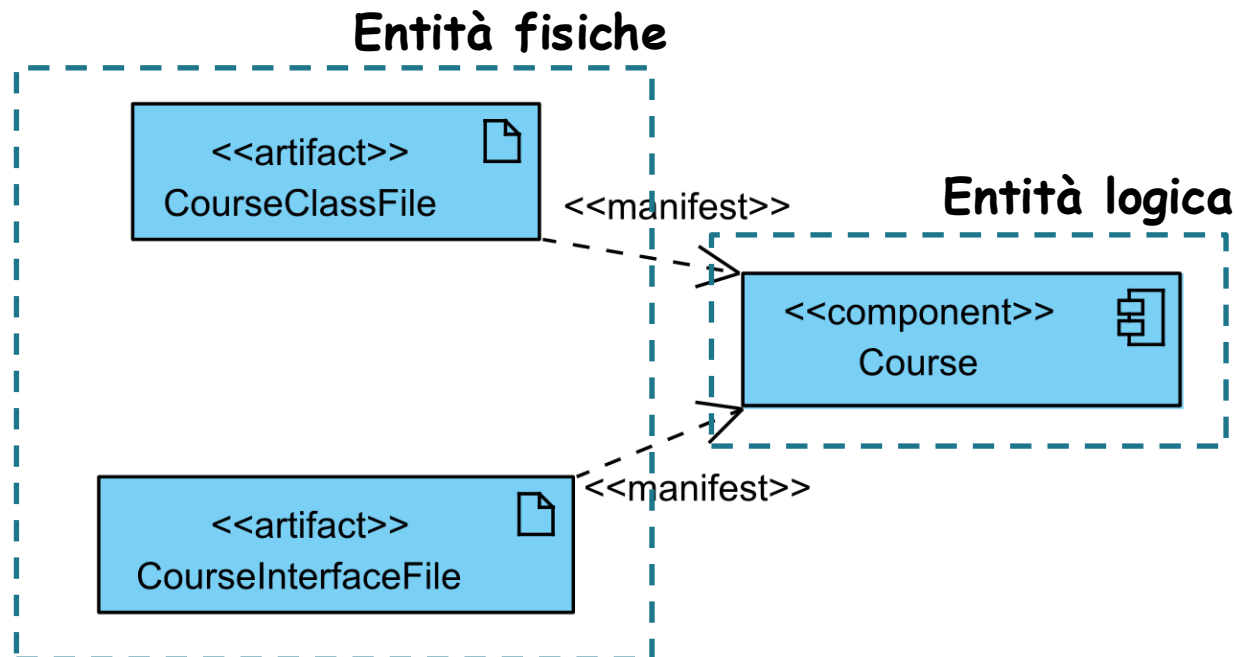
ARTEFATTI

- Sono **entità concrete** del mondo reale
- Per esempio:
 - **file di codice sorgente, file eseguibili, script, tabelle in un database, pagine HTML ...**
- Possono essere **dislocati** ('deployati') sui nodi
- Esiste la relazione di dipendenza tra artefatti

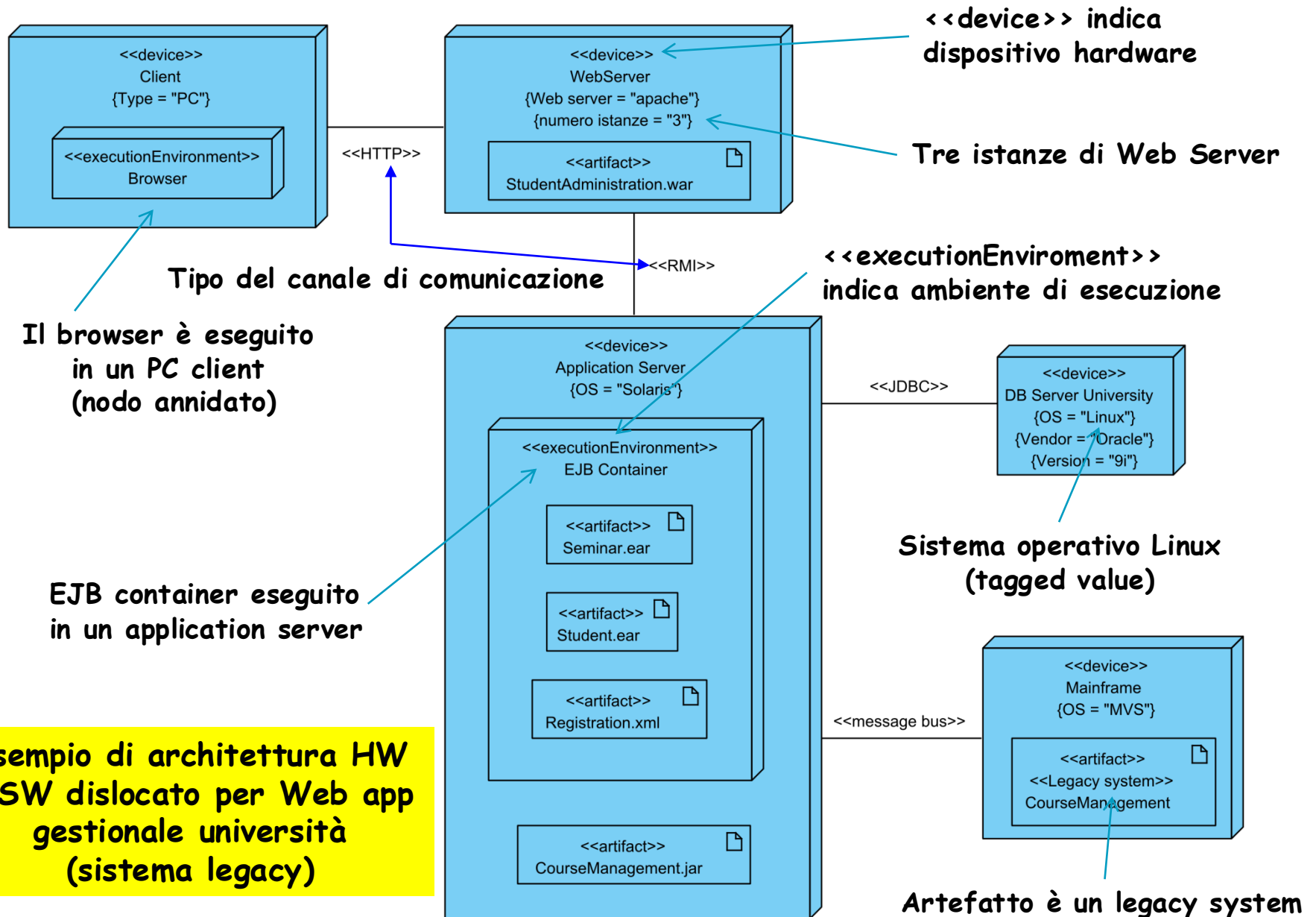


MANIFEST

- La relazione “**manifest**” indica che gli artefatti sono **rappresentazioni/manifestazioni fisiche** dei componenti
 - Per esempio: un componente può includere una classe e un'interfaccia, realizzate da due file che contengono il codice sorgente



NODI ANNIDATI E TAGGED VALUES



ESEMPIO/ESERCIZIO: OFFICINA MECCANICA

- Si supponga di dover decidere l'architettura software (cioè le **componenti**) e hardware (cioè i **nodi**) per un software che gestisce un **officina meccanica**
- Due tipi di utilizzatori:
 - **Meccanici** che devono richiedere i pezzi di ricambio per ogni intervento
 - Operazione: **richiesta pezzi**
 - **Magazzinieri** che devono fornire i pezzi
 - Ma devono anche tenere il magazzino sempre con le dovute scorte facendo le richieste ai fornitori quando i pezzi scarseggiano
 - Operazione: **carico e scarico pezzi dal magazzino**

COME DOVREBBE FUNZIONARE IL SOFTWARE?

Un meccanico ha associato una lista di interventi.
Dopo avere scelto un intervento, il meccanico identifica le operazioni da effettuare sul veicolo e i pezzi di ricambio necessari per l'intervento.
Inserisce la richiesta pezzi **nel sistema** e la spedisce al magazziniere



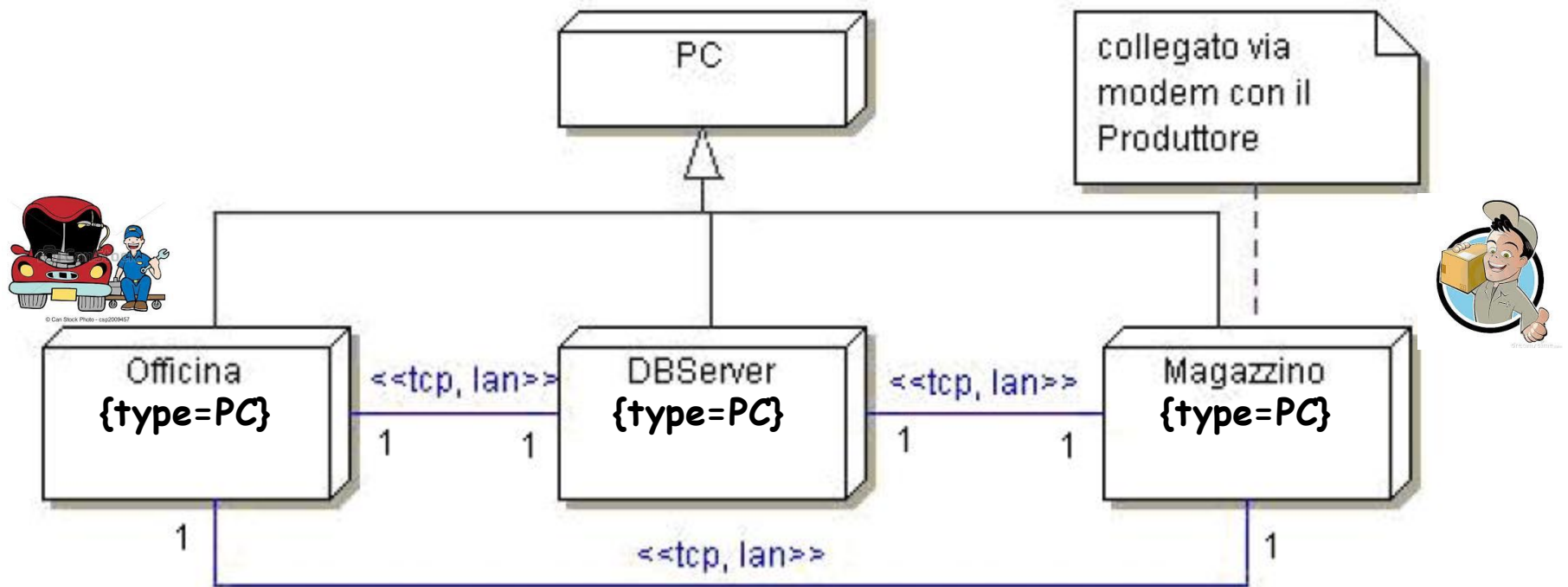
© Can Stock Photo - csp2009457



© Dreamstime

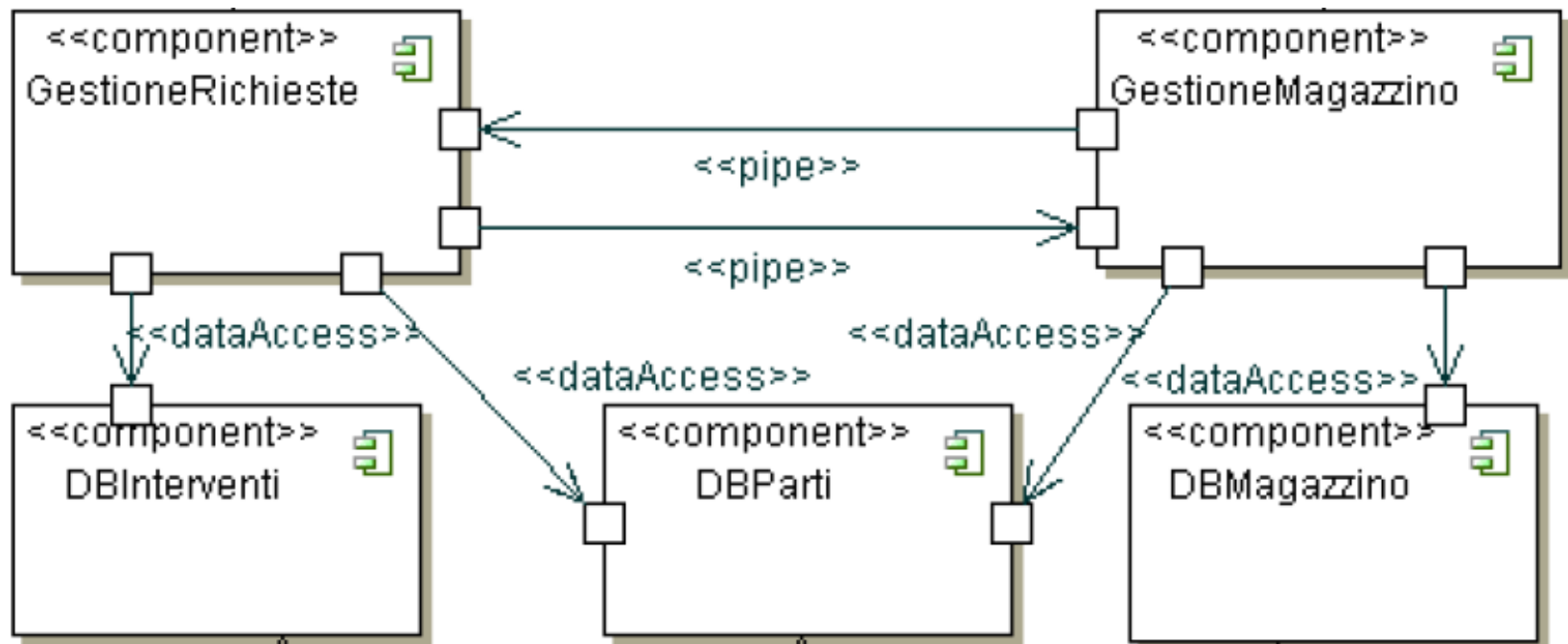
Il magazziniere periodicamente controlla la lista di richieste pezzi.
Ne seleziona una, verifica la disponibilità dei pezzi e se ci sono:
prepara i ricambi, li scarica dal magazzino e avverte il meccanico.
Se le scorte sono basse o mancano i pezzi fa le richieste ai fornitori

POSSIBILE ARCHITETTURA HARDWARE (DEPLOYMENT DIAGRAM)



'Tre Personal Computer in rete ...'

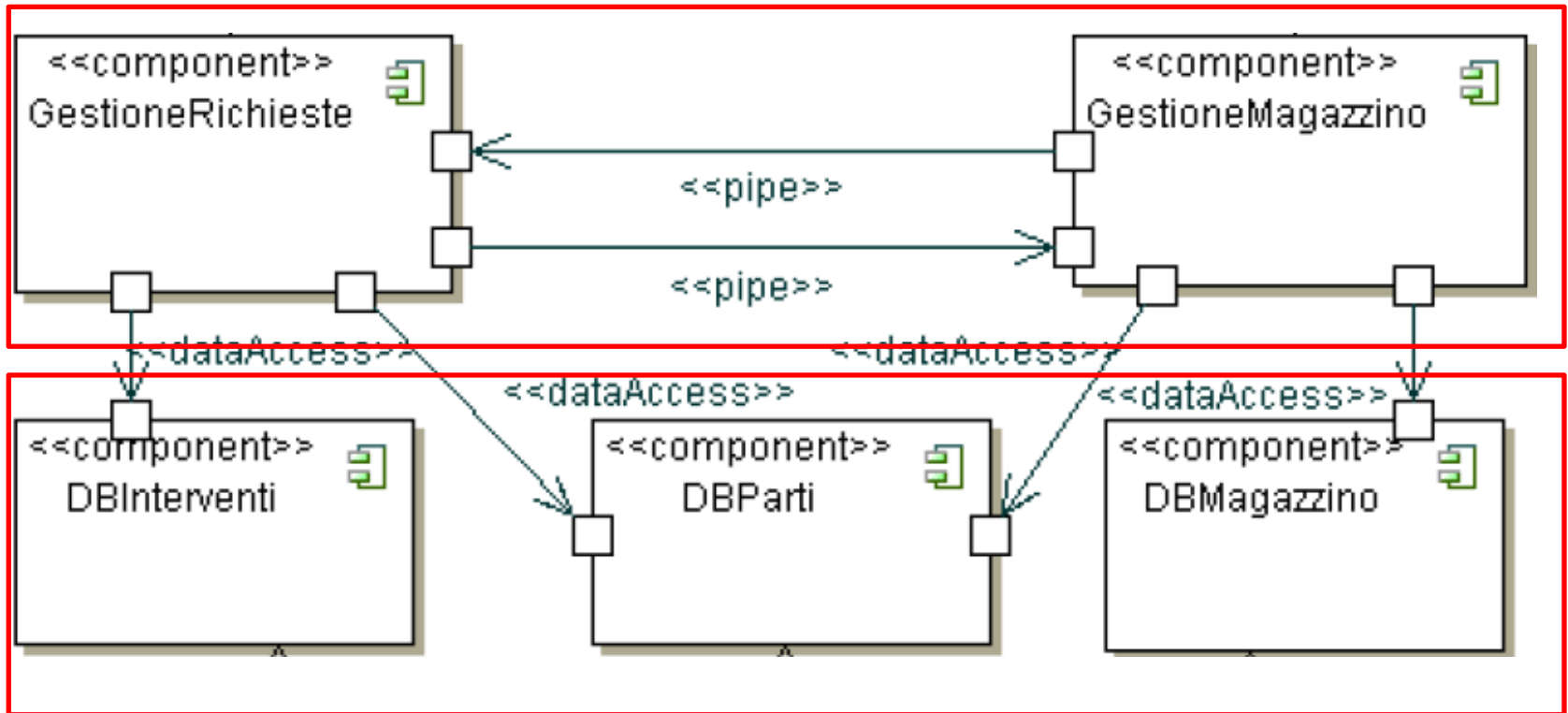
POSSIBILE ARCHITETTURA SOFTWARE (COMPONENT DIAGRAM)



POSSIBILE ARCHITETTURA SOFTWARE (COMPONENT DIAGRAM)

Business logic + GUI

Data logic

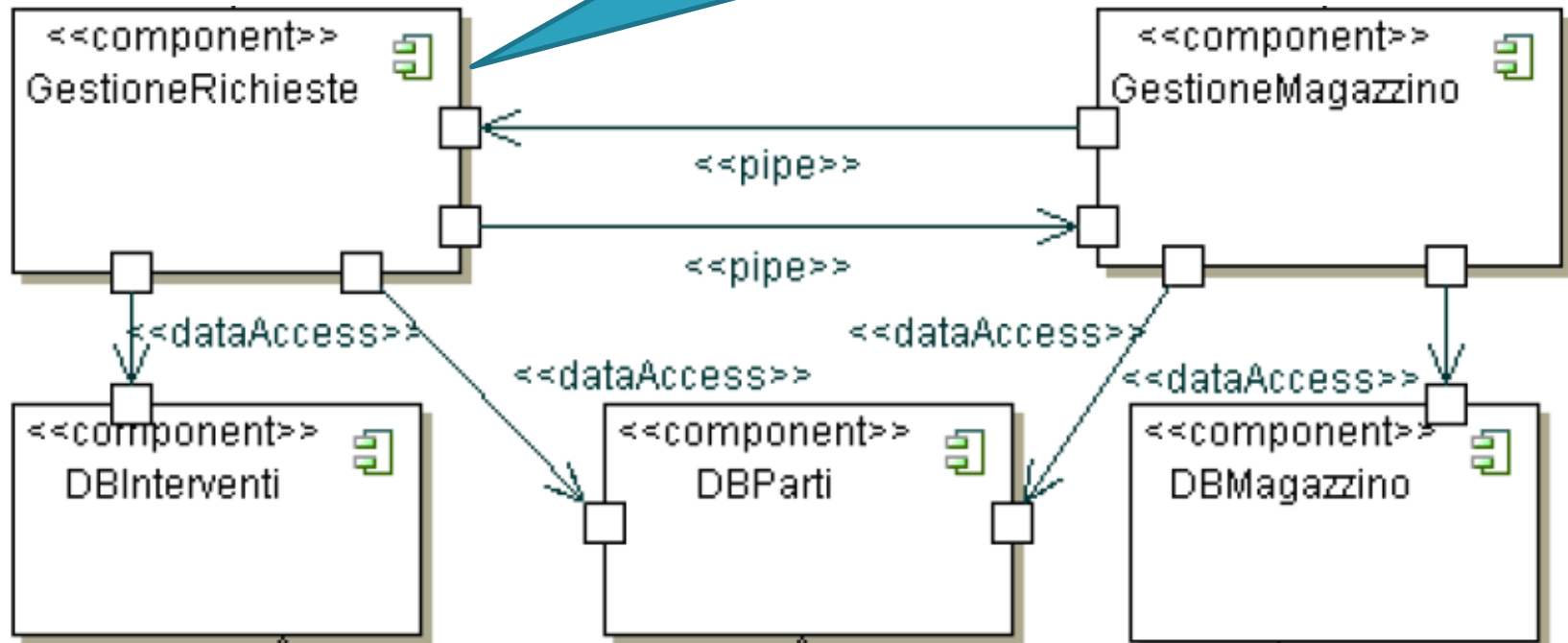


Gestore dati

POSSIBILE ARCHITETTURA SOFTWARE (COMPONENT DIAGRAM)



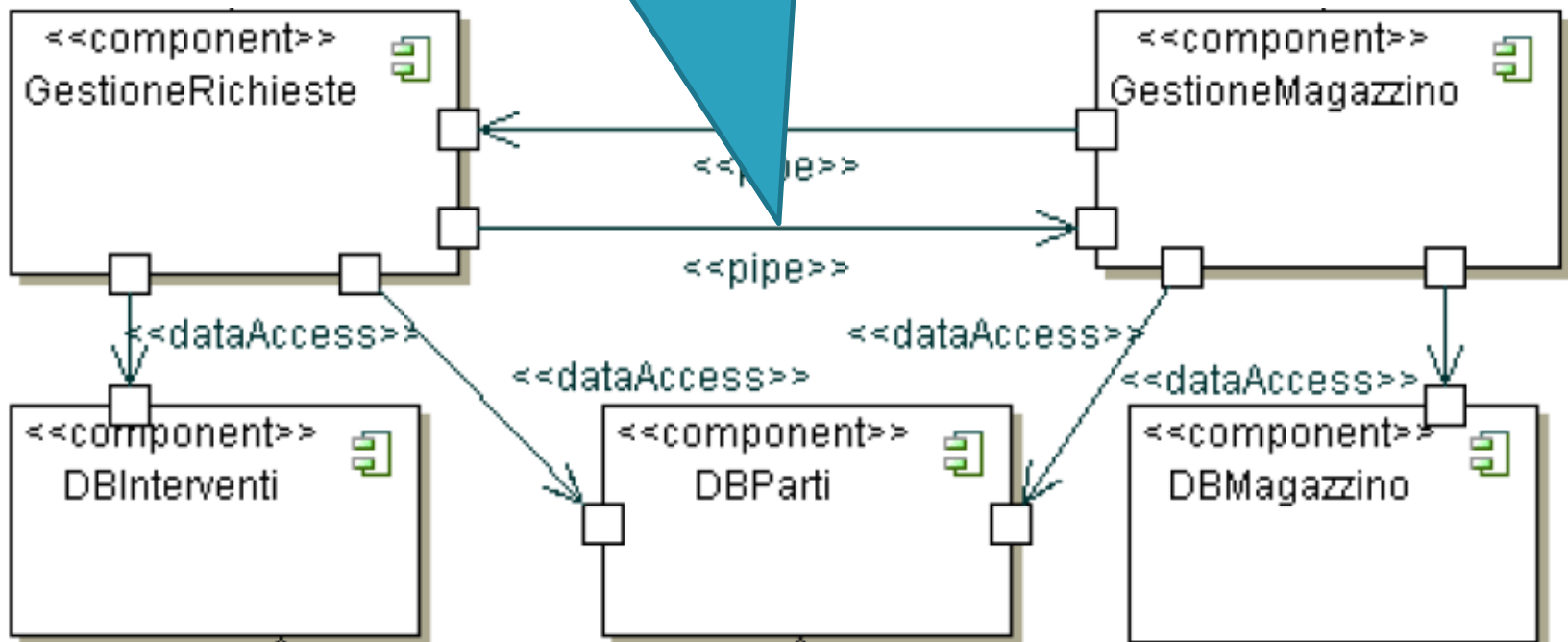
Gestisce le richieste dei pezzi
da parte dei meccanici
(sia GUI che Business logic)



POSSIBILE ARCHITETTURA SOFTWARE (COMPONENT DIAGRAM)

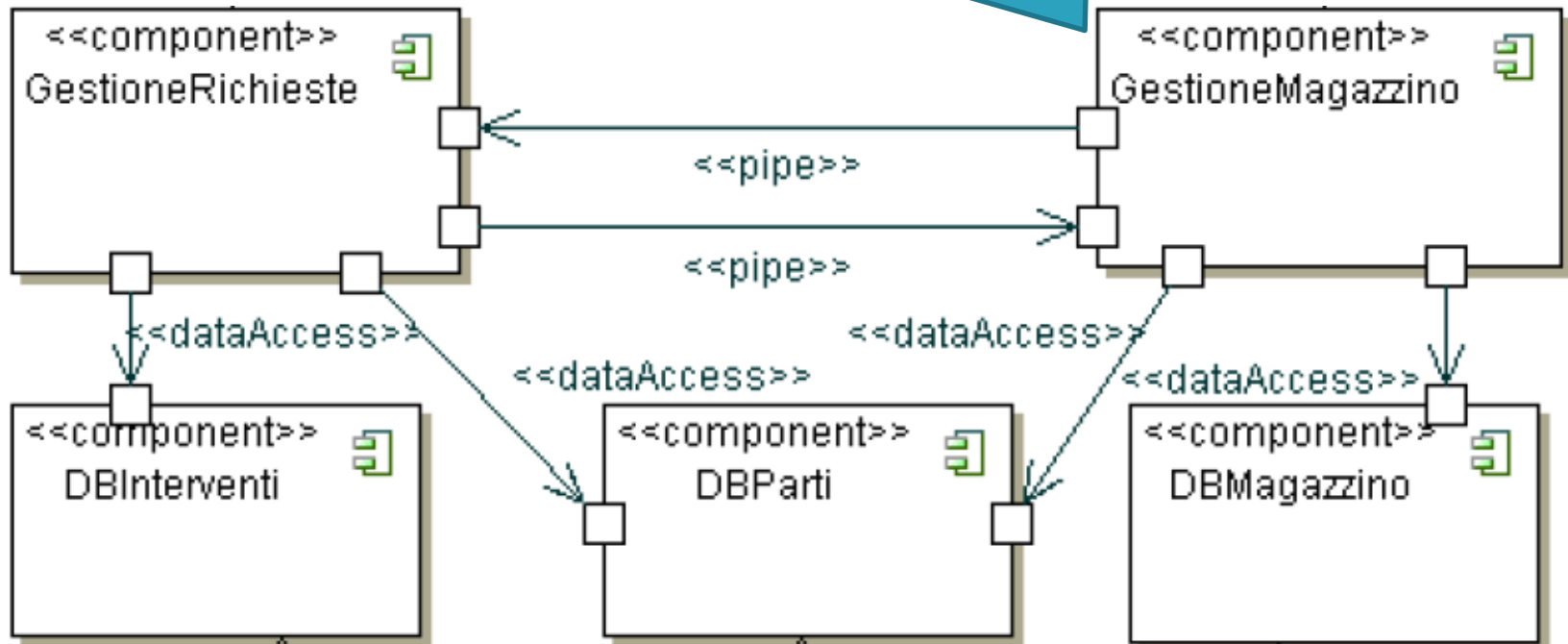


Invia le richieste dei pezzi a
GestioneMagazzino

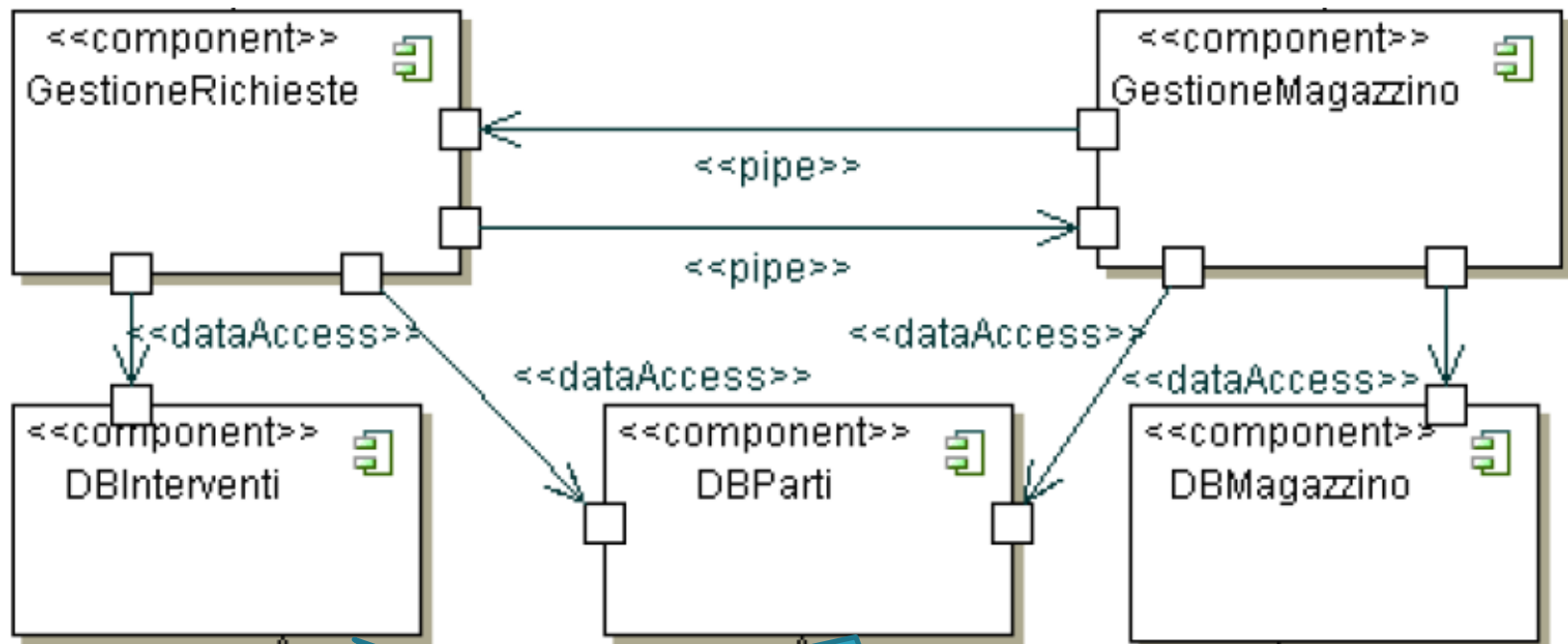


POSSIBILE ARCHITETTURA SOFTWARE (COMPONENT DIAGRAM)

Gestisce operazioni di carico e
scarico magazzino
(sia GUI che Business logic)



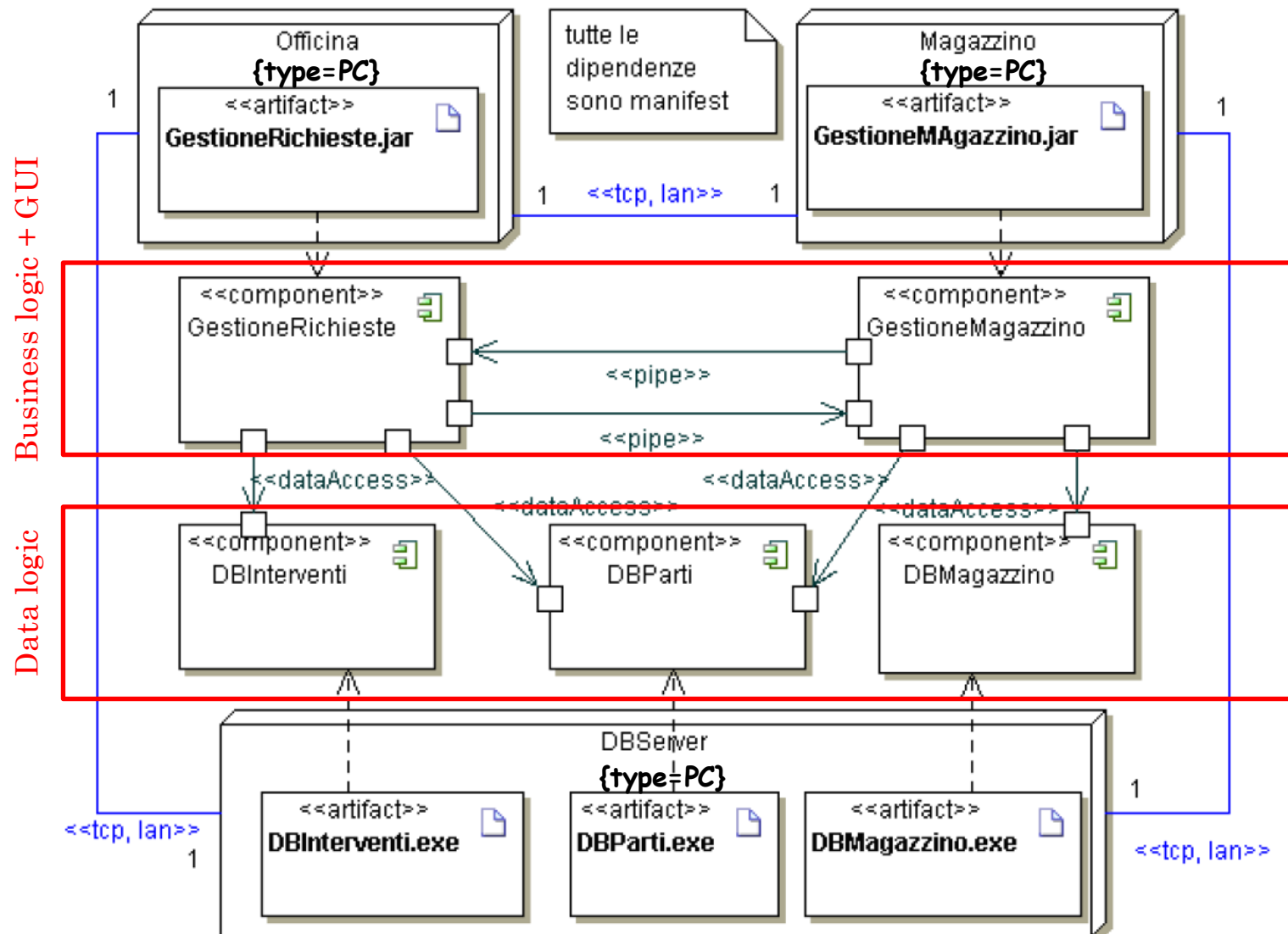
POSSIBILE ARCHITETTURA SOFTWARE (COMPONENT DIAGRAM)

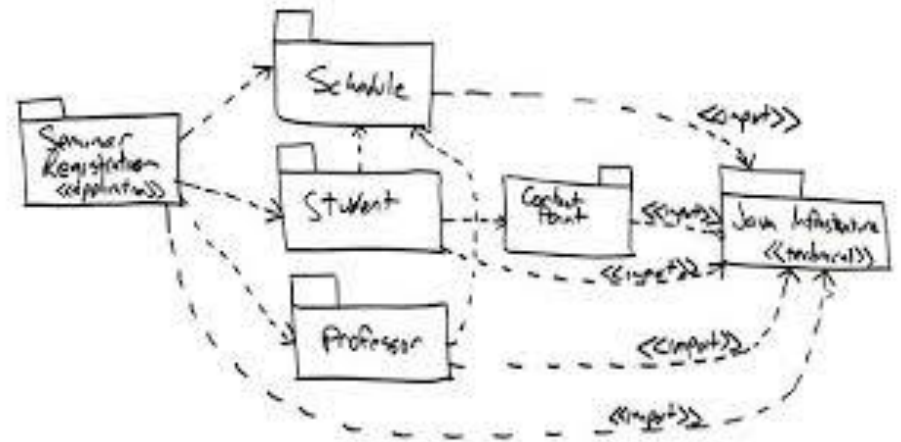


Gestiscono la persistenza (e il recupero) degli interventi, pezzi di ricambio e quantità in magazzino

COMPONENT + DEPLOYMENT DIAGRAM

Spesso può essere utile integrare i due diagrammi ...





PACKAGE DIAGRAM

PACKAGE

Library Domain

+ Catalog
+ Patron
+ Librarian
- Account

- Un package (in UML) è un **costrutto** che permette di prendere un numero arbitrario di elementi UML e **raggrupparli** insieme
 - Elementi UML = classi, casi d'uso, modelli UML, ...
 - Noi principalmente li vedremo come contenitori di classi
- Un package può contenere sia elementi (es. classi) che sotto-package
 - Gerarchie di package
- Concetto molto simile ai package nei linguaggi di programmazione (es. Java)
 - In UML: **“concetto + generale”**

NAMESPACE

- Ogni package definisce un **namespace**
 - **Regione** all'interno del quale tutti i nomi devono essere univoci
- Lo scopo dei namespace è quello di evitare confusione ed equivoci nel caso siano necessarie molte entità con nomi simili
- Per indicare la classe a cui mi sto riferendo occorre usare un **nome completamente qualificato**

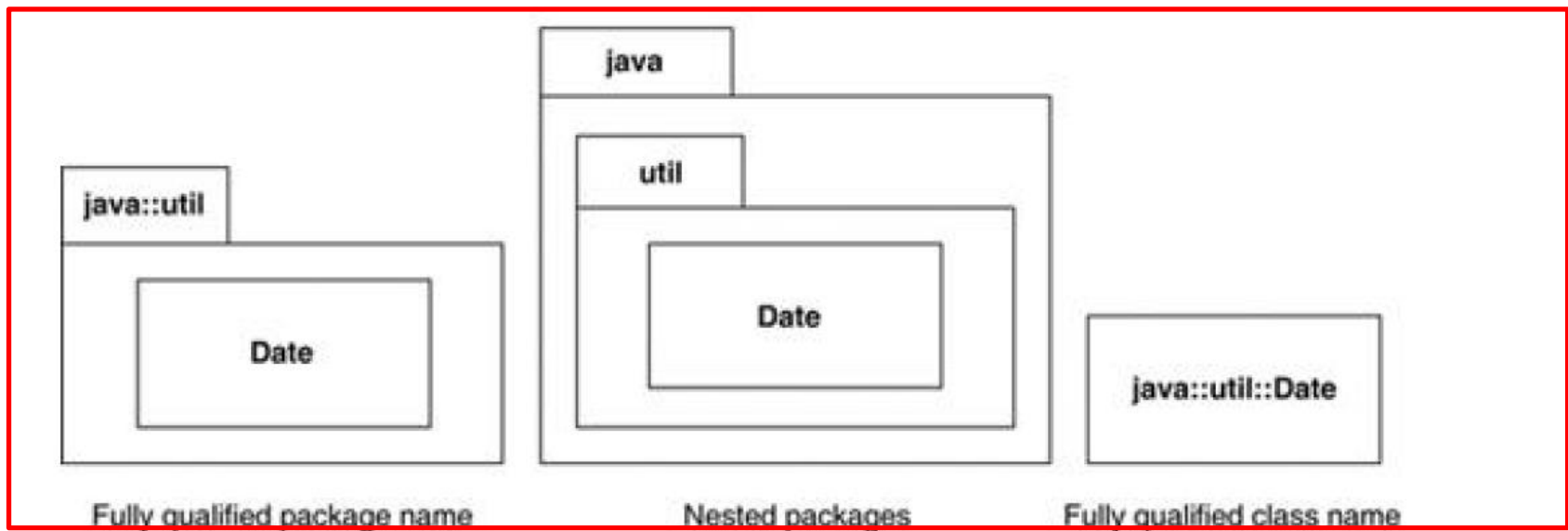
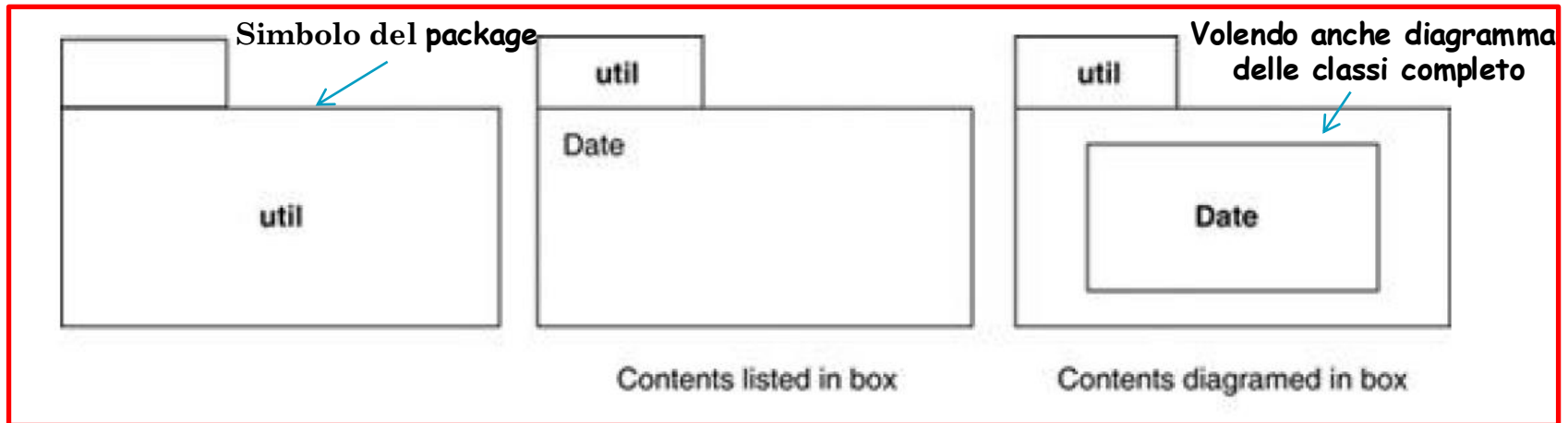
System::Data



Notazione UML

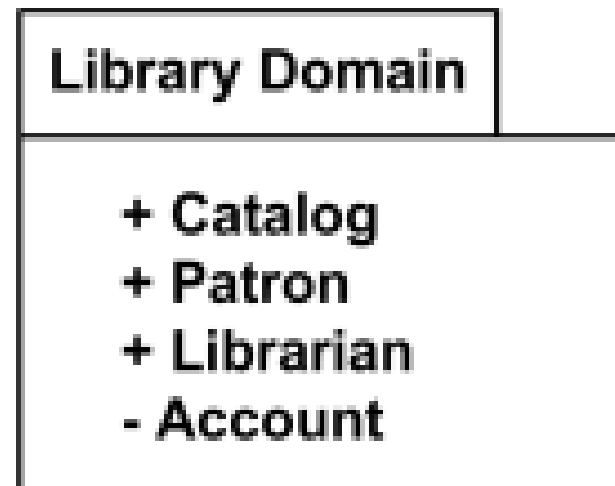
MartinFowler::Util::Data

MODI DI RAPPRESENTARE I PACKAGE



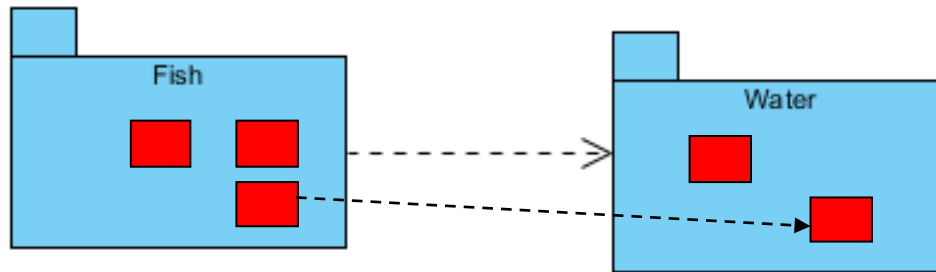
VISIBILITÀ

- Gli elementi contenuti in un package possono avere **una visibilità che indica se gli elementi sono visibili o meno ai clienti del package**
 - + = pubblica
 - - = privata
- Gli elementi visibili formano l'interfaccia del package



PACKAGE DIAGRAM (DIPENDENZE)

- I **Package diagram** descrivono i package e le dipendenze



- **Fish dipende da Water:** vuol dire che nel package Fish esiste almeno una classe che dipende da una classe che è nel package Water ...
 - La classe in Water deve essere visibile!
- Dipendenze tra package riassumono quelle tra gli elementi contenuti

COME SUDDIVIDERE LE CLASSI IN PACKAGE?

- Molto difficile!

- Occorre molta esperienza
- E' un compromesso!



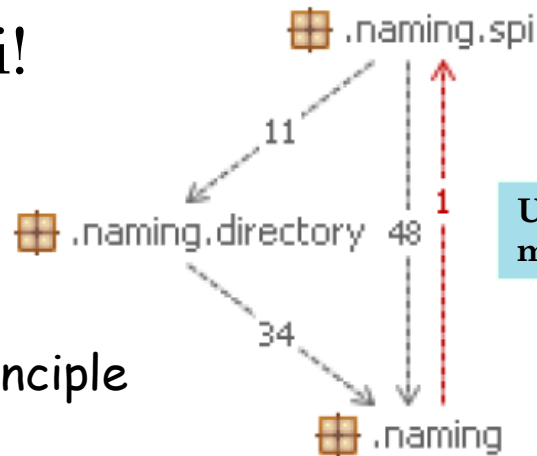
- Seguire i **principi di buona progettazione**:

- **High cohesion and low coupling**
- **Common Reuse Principle**: le classi in un package dovrebbero essere sempre riusate assieme
- **Acyclic Dependency Principle**: no cicli!
- ...



CICLI E DIPENDENZE ENTRANTI

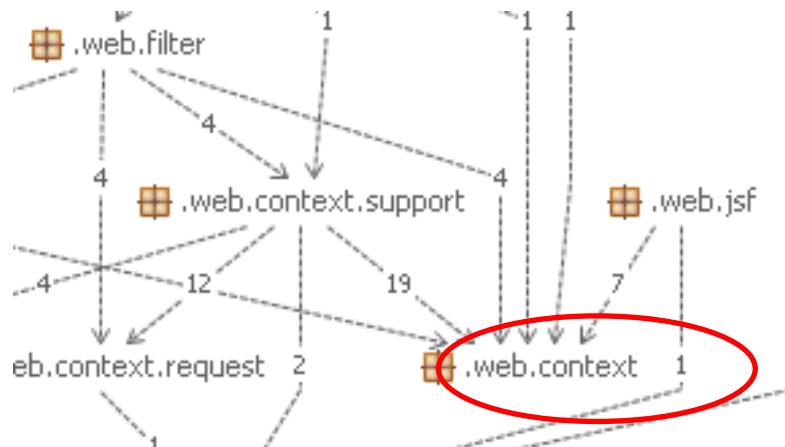
- Attenzione ai cicli!



Una modifica potrebbe comportare modifiche a cascata ...

Acyclic Dependency principle

- Attenzione ai package con molte dipendenze entranti!

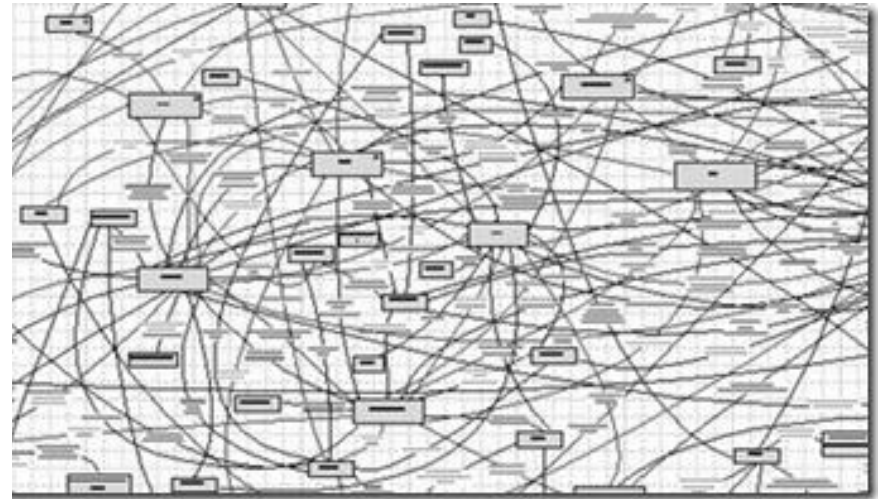
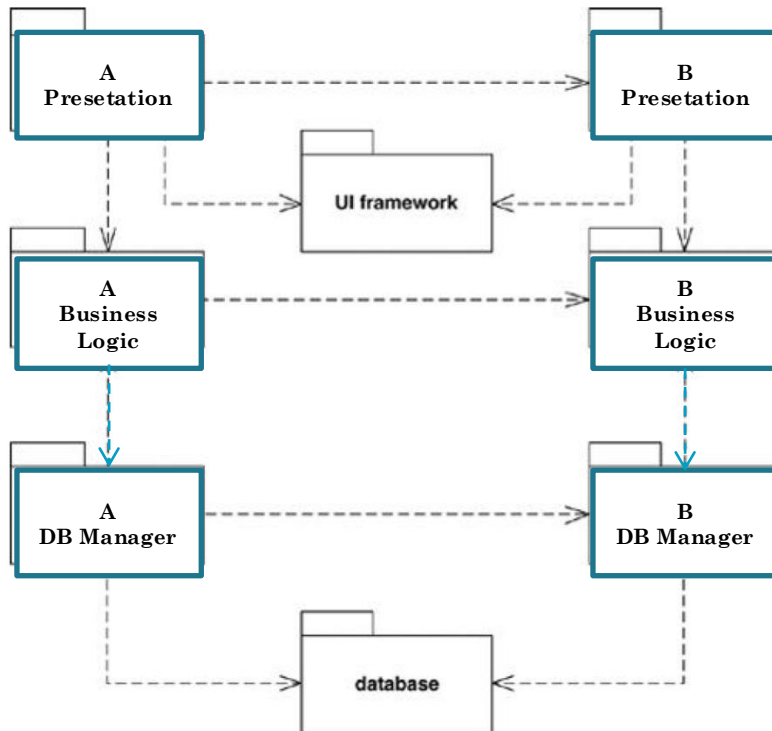


Una modifica a web.content potrebbe impattare molti altri package!

PACKAGE DIAGRAM



- “In un sistema medio-grande, un diagramma dei package è una delle **cose più utili** per tenere sotto controllo la complessità strutturale del codice”



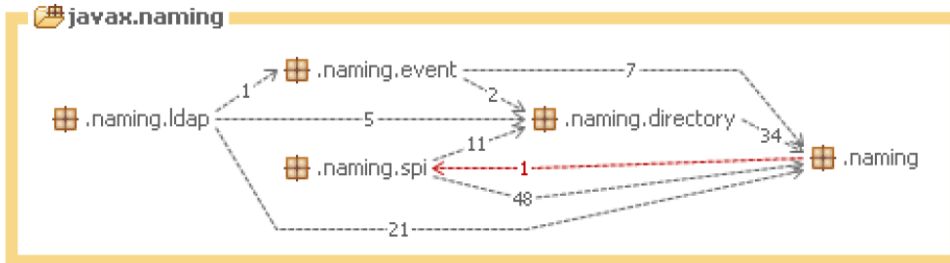
COME FARE IN PRATICA?

- Progettare l'applicazione cercando di seguire il più possibile i **principi di buona progettazione**
- Generare il **diagramma dei package** a partire dal codice stesso usando tool specifici (**reverse engineering**)
 - Es. Stan4J
- Identificare cicli, package poco coesi e package con tante dipendenze entranti
 - Provare a migliorare le cose



RISOLUZIONE DEI CICLI

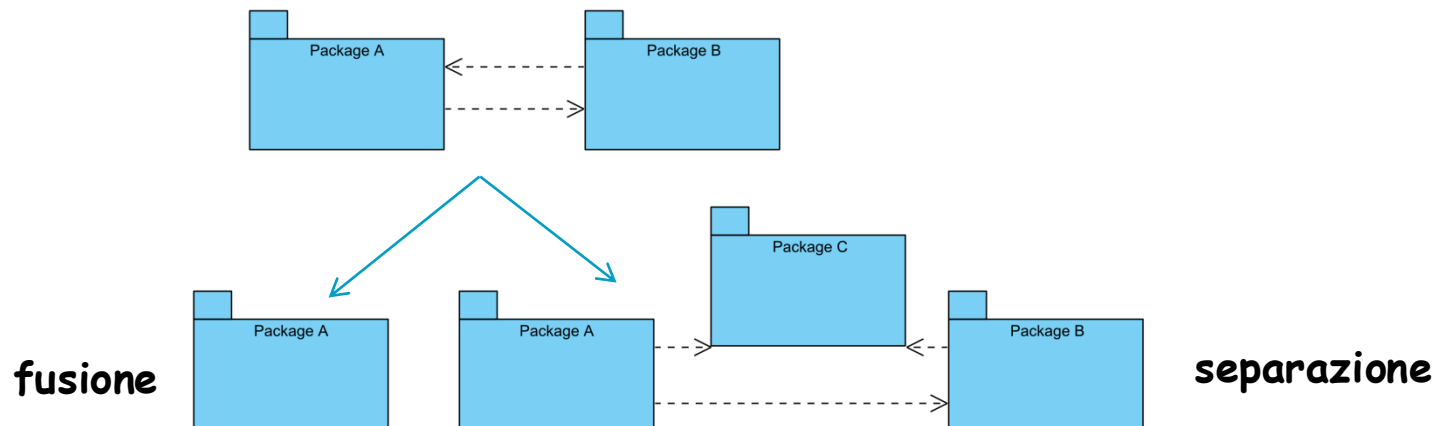
- **Eliminare** una o più dipendenze (se possibile)



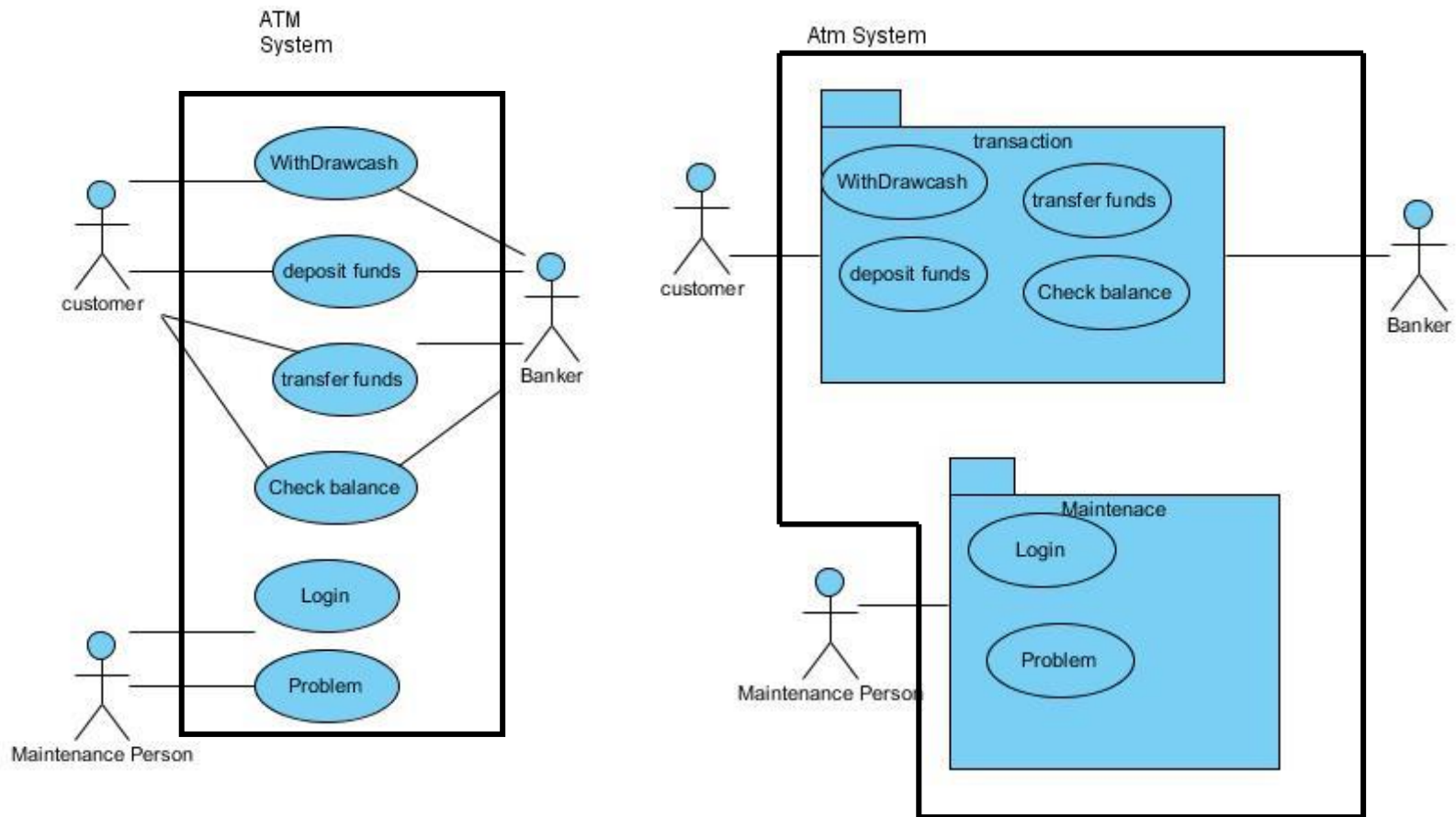
Stan4J ci indica la strada più breve

- **Fusione**

- **Separazione**. Si tenta di estrarre le entità che creano il ciclo e si spostano in un altro package



NON SOLO CLASSI: ESEMPIO USE CASE DIAGRAM



Use case Diagram



Use case **Package** Diagram

RIASSUMENDO

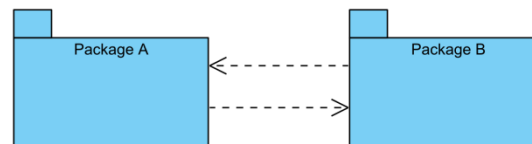
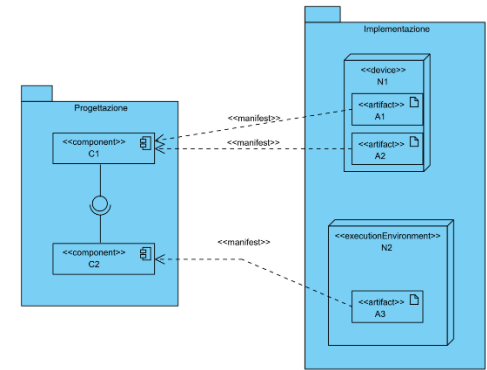
- Visti tre tipi di diagrammi strutturali:

- Usati per scopi diversi

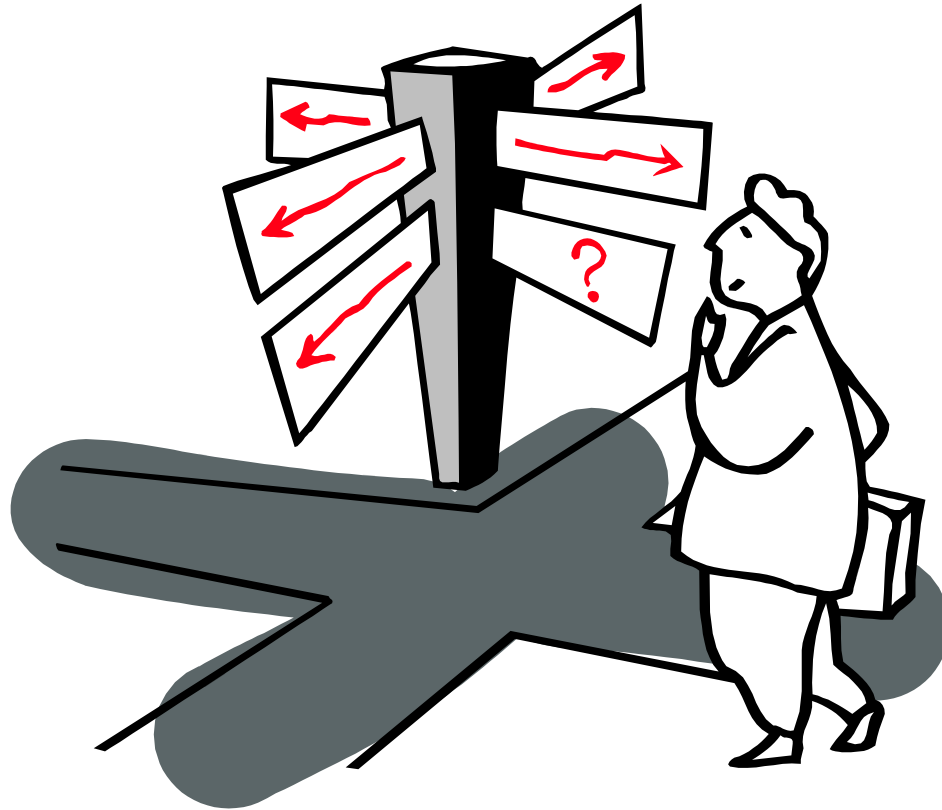
- Component diagram → describe l'architettura software
- Deployment diagram → describe la relazione tra HW e SW
- Package diagram → “raggruppa elementi UML”

- Usati in momenti diversi dello sviluppo

- Component diagram → design dell'architettura
- Deployment diagram → implementazione/deployment
- Package diagram → dipende da cosa raggruppiamo ...



THE END ...



Domande?